

数据结构

Data Structures & Algorithms

MS Roslyn

2026 年 10 月 9 日

摘要

本教程系统介绍数据结构的核心概念与经典算法，涵盖位置记数系统、数据系统与物理存储、线性结构与抽象数据类型、树与二叉树、图论，以及算法定义与渐近分析、经典算法与核心运算，并通过不同算法的对比帮助读者理解时间与空间复杂度之间的权衡。教程将抽象数据类型与具体实现相结合，为程序设计提供坚实的数据组织基础。

目录

第一章	位置记数系统	6
1.1	任意进制定义	7
1.1.1	基数与合法数码集 (Alphabet of Digits)	7
1.1.2	离散数码序列 (Numeral Sequence)	7
1.1.3	按权展开映射 (Polynomial Expansion)	7
1.2	任意进制转换定义 ($n \rightarrow m$)	8
1.2.1	转换核心公理: 值守恒 (Value Conservation)	8
1.2.2	数学求解算子: 数码分离 (Digit Extraction Operator)	8
1.3	数学算子应用示例 ($10 \rightarrow 2$ 进制)	8
1.3.1	整数部分求解 ($y = 11$, 求解 $j \geq 0$ 的数码)	8
1.3.2	小数部分求解 ($y_f = 0.625$, 求解 $j < 0$ 的数码)	8
1.3.3	最终值守恒合并	9
1.4	后继与进位 (Successor and Carry)	9
1.5	列式加法 (Column Addition)	10
1.6	闭式到递推的细化 (进制转换的迭代实现)	10
1.6.1	整数项递推: 商与余数 (Division-Remainder Iteration)	10
1.6.2	小数项递推: 积与整数 (Multiplication-Integer Iteration)	11
第二章	数据系统与物理存储	12
2.1	数据、数据元素与数据对象	13
2.2	逻辑结构与存储结构的关系	14
2.3	四种基本逻辑结构	15
2.4	四种存储映像	16
2.4.1	顺序存储 (Sequential Storage)	16
2.4.2	链式存储 (Linked Storage)	16
2.4.3	索引存储 (Indexed Storage)	16
2.4.4	散列存储 (Hash Storage)	17
2.4.5	四种映像的对比	17
2.5	顺序存储的地址计算	18
2.5.1	一维数组	18
2.5.2	二维数组的行优先与列优先	18
2.5.3	完整数值例题	18
2.6	索引存储与散列存储的基本思想	19
2.7	抽象数据类型 (ADT)	19

2.8	本章小结与常见误区	20
第三章	线性结构与抽象数据类型	21
3.1	物理存储结构：线性表 (Linear List)	22
3.1.1	顺序存储结构 (顺序表 / 数组)	22
3.1.2	链式存储结构 (链表 / Link List)	22
3.1.3	线性表操作规约	23
3.2	核心矛盾对比：顺序表 vs 链表	23
3.3	栈 (Stack)	24
3.3.1	代数定义	24
3.3.2	物理实现方案	24
3.4	队列 (Queue)	25
3.4.1	代数定义	25
3.4.2	循环队列 (Circular Queue)	25
3.5	双端队列 (Deque)	26
3.6	字符串 (String)	27
3.6.1	形式化定义	27
3.6.2	串匹配算子 (KMP)	28
3.6.3	BM 与 Rabin-Karp	29
3.7	高级映射与哈希表体系	30
3.7.1	字典 (Dictionary) / 映射 (Map)	30
3.7.2	哈希表 (Hash Table)	30
3.7.3	哈希碰撞 (Hash Collision)	30
3.8	字典树 (Trie) 与前缀检索	32
第四章	非线性结构 I：树与二叉树	33
4.1	通用树的基础属性	34
4.2	二叉树 (Binary Tree) 及其形态特征	34
4.3	树的物理存储	35
4.4	树的遍历 (Tree Traversal)	36
4.4.1	递归遍历 (Recursive Traversal)	36
4.4.2	非递归遍历 (Iterative Traversal)	37
4.4.3	线索二叉树 (Threaded Binary Tree)	37
4.4.4	遍历序列与树的唯一性	38
4.5	二叉搜索树 (BST)	39
4.6	平衡二叉树 (AVL)	40
4.7	红黑树 (Red-Black Tree)	40
4.8	B 树与 B+ 树 (B-Tree / B+ Tree)	42
4.9	伸展树 (Splay Tree)	43
4.10	Huffman 最优树	44
4.11	堆体系 (Heap)	45
4.12	树上查询：LCA 倍增	46

第五章 非线性结构 II: 图论	47
5.1 图的形式化定义	48
5.2 边的拓扑分类	48
5.3 连通性深度分析	48
5.4 割点与双连通	49
5.5 图的遍历与降维	50
5.5.1 遍历规约	50
5.5.2 拓扑排序	51
5.6 图的物理存储实现	51
第六章 算法定义与渐近分析	52
6.1 算法的定义与五个特性	53
6.1.1 从直观到形式化	53
6.1.2 五个基本特性	53
6.1.3 算法与程序的区别	53
6.2 问题规模与基本操作	54
6.2.1 问题规模	54
6.2.2 基本操作	54
6.2.3 最好、最坏与平均情况	54
6.3 渐进记号	55
6.3.1 五种记号的形式化定义	55
6.3.2 上界常数的几何意义	55
6.3.3 常见复杂度阶的排序	56
6.4 算法分析的基本法则	57
6.4.1 加法法则与乘法法则	57
6.4.2 代码片段复杂度例题	57
6.5 递归算法的复杂度分析	58
6.5.1 递推式与递归树	58
6.5.2 代入法	59
6.5.3 主定理 (Master Theorem)	59
6.5.4 主定理例题	59
6.5.5 归并排序的算法与复杂度	60
6.6 空间复杂度	60
6.7 本章小结与常见误区	61
6.7.1 小结	61
6.7.2 常见误区	61
第七章 动态经典算法与核心算子	62
7.1 顶层设计范式	63
7.1.1 分治策略 (Divide and Conquer)	63
7.1.2 贪心算法 (Greedy)	63
7.1.3 动态规划 (Dynamic Programming)	63
7.2 排序算法算子 (Sort)	65

7.2.1	比较类排序	65
7.2.2	非比较类排序	66
7.3	查找与并查集	68
7.3.1	二分查找 (Binary Search)	68
7.3.2	插值查找 (Interpolation Search)	68
7.3.3	分块查找 (Block Search)	69
7.3.4	并查集 (Union-Find)	69
7.4	图论核心算法	70
7.4.1	最短路径算法	70
7.4.2	最小生成树 (MST)	71
7.4.3	拓扑排序: Kahn 算法	72
7.4.4	启发式搜索: A* (A-Star)	72
7.5	高级工程专项算子	73
7.5.1	网络流 (Network Flow)	73
7.5.2	二分图最大匹配	74
7.5.3	强连通分量 (SCC): Tarjan 算法	75
第八章	算法对比	76
8.1	为什么要比较算法: 多维权衡	77
8.1.1	四个权衡维度	77
8.1.2	渐近界与常数因子	77
8.2	排序算法综合对比	78
8.2.1	比较类排序的下界	78
8.2.2	分治排序的递推	78
8.2.3	非比较排序的线性界	78
8.3	查找算法对比	79
8.3.1	平均查找长度	79
8.4	增长曲线与计数级差异	80
8.4.1	增长曲线的可视化	80
8.4.2	计数级差异算例 ($n = 10^5$)	80
8.5	时间-空间权衡	81
8.5.1	原地与非原地	81
8.5.2	以空间换时间的边界	81
8.6	如何选择算法: 决策清单	81
8.6.1	排序算法选择	81
8.6.2	查找算法选择	82
8.7	其他算子横向对比	82
8.7.1	高级图论算子	82
8.7.2	树算子	83
8.8	本章小结	83
8.9	常见误区	83

第一章 位置记数系统

在数学中， n 进制（进位计数制，Positional Numeral System）是指用一组固定的符号来表示任意实数的计数方法。其严谨的数学形式化定义由以下三个核心要素构成：

1.1 任意进制定义

1.1.1 基数与合法数码集 (Alphabet of Digits)

- 对象：基数 $n \in \mathbb{Z}^+, n > 1$ 。
- 不变式：有限数码集合

$$S_n = \{x \mid x \in \mathbb{N}, 0 \leq x < n\} = \{0, 1, 2, \dots, n-1\}$$

满足 $|S_n| = n$ 且 $\forall d \in S_n, 0 \leq d < n$ 。

1.1.2 离散数码序列 (Numeral Sequence)

- 对象：双向无穷序列 $\{P_i\}_{i=-\infty}^{\infty}$ ，约束 $\forall i \in \mathbb{Z}, P_i \in S_n$ 。
- 不变式（唯一性）：为使 $y \mapsto \{P_i\}$ 唯一，禁止两类尾：最高有效位左侧不含无穷个 0，小数末尾不含无穷个 $(n-1)$ （排除 $0.999\dots = 1.000\dots$ 二义性）。
- 定理：在上述不变式下， $\{P_i\} \mapsto y$ 为双射（除零测集外）。

1.1.3 按权展开映射 (Polynomial Expansion)

- 对象：解释映射 $\mathcal{I}_n : S_n^{\mathbb{Z}} \rightarrow \mathbb{R}$,

$$y = \mathcal{I}_n(\{P_i\}) = \sum_{i=-\infty}^{\infty} P_i \cdot n^i$$

展开为 $y = \dots + P_2 n^2 + P_1 n^1 + P_0 n^0 + P_{-1} n^{-1} + P_{-2} n^{-2} + \dots$ 。

- 不变式：整数部分 $i \geq 0$ 与小数部分 $i < 0$ 同一公式分区，仅指数符号不同。

2^3	2^2	2^1	2^0	$\cdot$	2^{-1}	2^{-2}	2^{-3}
1	0	1	1	$\cdot$	1	0	1

$$y = \sum_i P_i 2^i = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = 11.625$$

图 1.1: 按权展开：位权与数码一一对应 $((1011.101)_2 = 11.625)$

1.2 任意进制转换定义 ($n \rightarrow m$)

1.2.1 转换核心公理：值守恒 (Value Conservation)

- 对象：源序列 $\{P_i\} \in S_n^{\mathbb{Z}}$ ，目标序列 $\{Q_j\} \in S_m^{\mathbb{Z}}$ 。

- 不变式（值守恒）：

$$y = \sum_i P_i n^i = \sum_j Q_j m^j, \quad P_i \in S_n, Q_j \in S_m$$

转换是数码表示的变换，实轴值 y 不变。

- 定理：在唯一性不变式下，满足值守恒的 $\{Q_j\}$ 存在唯一。

1.2.2 数学求解算子：数码分离 (Digit Extraction Operator)

- 对象：已知 y ，求 Q_j 的闭式算子（全局算子）：

– 整数位 $j \geq 0$: $Q_j = \lfloor y/m^j \rfloor \bmod m$ 。

– 小数位 $j < 0$: $Q_j = \lfloor y_{\text{fraction}} \cdot m^{-j} \rfloor \bmod m$ ，其中 $y_{\text{fraction}} = y - \lfloor y \rfloor \in [0, 1)$ 。

- 不变式： $\forall j, Q_j \in S_m$ 。

1.3 数学算子应用示例 ($10 \rightarrow 2$ 进制)

【示例】将 $y = 11.625$ 转换为 $m = 2$ ，验证值守恒。

1.3.1 整数部分求解 ($y = 11$, 求解 $j \geq 0$ 的数码)

- $j = 0$: $Q_0 = \lfloor 11/2^0 \rfloor \bmod 2 = 1$
- $j = 1$: $Q_1 = \lfloor 11/2^1 \rfloor \bmod 2 = \lfloor 5.5 \rfloor \bmod 2 = 1$
- $j = 2$: $Q_2 = \lfloor 11/2^2 \rfloor \bmod 2 = 0$
- $j = 3$: $Q_3 = \lfloor 11/2^3 \rfloor \bmod 2 = 1$
- $j = 4$: $\lfloor 11/2^4 \rfloor = 0$ 终止。得 $(11)_{10} = (1011)_2$ ，即 $Q_3 Q_2 Q_1 Q_0$ 。

1.3.2 小数部分求解 ($y_f = 0.625$, 求解 $j < 0$ 的数码)

- $j = -1$: $Q_{-1} = \lfloor 0.625 \cdot 2^1 \rfloor \bmod 2 = 1$
- $j = -2$: $Q_{-2} = \lfloor 0.625 \cdot 2^2 \rfloor \bmod 2 = 0$
- $j = -3$: $Q_{-3} = \lfloor 0.625 \cdot 2^3 \rfloor \bmod 2 = 1$
- $j = -4$ 起全 0 终止。得 $(0.625)_{10} = (0.101)_2$ 。

1.3.3 最终值守恒合并

$$\sum_{j=-3}^3 Q_j 2^j = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = 11.625$$

故 $(11.625)_{10} = (1011.101)_2$ 。

1.4 后继与进位 (Successor and Carry)

- 对象：序列 $\{P_i\} \in S_n^{\mathbb{Z}}$ ，进位 $c_i \in \{0, 1\}$ 。
- 不变式：后继 $\text{succ} : S_n^{\mathbb{Z}} \rightarrow S_n^{\mathbb{Z}}$ 定义为 $y \mapsto y + 1$ 在数码层的实现。逐位方程：

$$s_i = P_i + c_i, \quad Q_i = s_i \bmod n, \quad c_{i+1} = \lfloor s_i/n \rfloor, \quad c_0 = 1$$

特例：若 $P_i = n - 1$ 则 $Q_i = 0, c_{i+1} = 1$ 继续；否则 $Q_i = P_i + 1, c_{i+1} = 0$ 终止。

- 定理： $\mathcal{I}_n(\text{succ}(\{P_i\})) = \mathcal{I}_n(\{P_i\}) + 1$ ，且进位链长度即首个非 $n - 1$ 的位置，最坏 $O(k)$ 。

Algorithm 1 后继（进位链）

```
1:  $i \leftarrow 0; c \leftarrow 1$ 
2: while  $c = 1$  do
3:    $s \leftarrow P_i + c; Q_i \leftarrow s \bmod n; c \leftarrow \lfloor s/n \rfloor; i \leftarrow i + 1$ 
4: end while
```

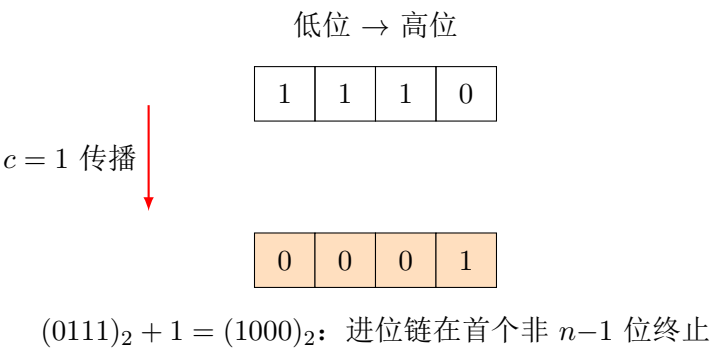


图 1.2: 后继：进位链最长 $O(k)$

1.5 列式加法 (Column Addition)

- 对象: $\{P_i\}, \{Q_i\} \in S_n^{\mathbb{Z}}$, 和 $\{R_i\} \in S_n^{\mathbb{Z}}$, 进位 $c_i \in \{0, 1\}$ 。
- 不变式: 逐列

$$s_i = P_i + Q_i + c_i, \quad R_i = s_i \bmod n, \quad c_{i+1} = \lfloor s_i/n \rfloor, \quad c_0 = 0$$

且 $\forall i, s_i \leq 2(n-1) + 1 = 2n-1 \implies c_{i+1} \in \{0, 1\}$ 。

- 定理 (正确性): $\sum_i R_i n^i = \sum_i P_i n^i + \sum_i Q_i n^i$; 若末进位 $c_k = 1$ 则 $R_k = 1$, 否则无溢出。

Algorithm 2 列式加法

```
1:  $c \leftarrow 0; i \leftarrow 0$ 
2: while  $i < k$  do
3:    $s \leftarrow P_i + Q_i + c; R_i \leftarrow s \bmod n; c \leftarrow \lfloor s/n \rfloor; i \leftarrow i + 1$ 
4: end while
5: if  $c = 1$  then  $R_k \leftarrow 1$ 
6: end if
```

$c_1=1 \ c_2=1 \ c_3=1 \ c_4=0$

1	1	1	0
1	0	0	0
0	0	0	1

低位在左: $(0111)_2 + (0001)_2 = (1000)_2$, 进位逐列向左传播

图 1.3: 列式加法: $s_i = P_i + Q_i + c_i$, $R_i = s_i \bmod n$, $c_{i+1} = \lfloor s_i/n \rfloor$

1.6 闭式到递推的细化 (进制转换的迭代实现)

直接计算闭式中的 m^j 需 $O(\log m^j)$ 位与高精度浮点, 易引入舍入误差。数学上将其细化为单步递推, 仅用 $O(1)$ 的取模/取整。

1.6.1 整数项递推: 商与余数 (Division-Remainder Iteration)

- 对象: 整数状态 $y_j \in \mathbb{N}$ 。

- 状态转移:

$$\begin{cases} Q_j = y_j \bmod m \\ y_{j+1} = \lfloor y_j/m \rfloor \end{cases}, \quad y_0 = \lfloor y \rfloor$$

- 不变式: $y = y_{j+1} \cdot m^{j+1} + \sum_{t=0}^j Q_t m^t$ 。
- 终止: $y_{j+1} = 0$ 。

核心原理解析

- $Q_j = y_j \bmod m$ 隔离最低位； $y_{j+1} = \lfloor y_j/m \rfloor$ 右移一位。
- 产出顺序为 $Q_0 \rightarrow Q_1 \dots$ （低位到高位），序列表示为 $\text{rev}(Q_0 \dots Q_{k-1})$ 。

1.6.2 小数项递推：积与整数 (Multiplication-Integer Iteration)

- 对象：小数状态 $y_j \in [0, 1)$ 。
- 状态转移：
$$\begin{cases} Q_j = \lfloor y_j \cdot m \rfloor \\ y_{j+1} = y_j \cdot m - Q_j \end{cases}, \quad y_{-1} = y - \lfloor y \rfloor$$
- 不变式： $y_{-1} = \sum_{t=j}^{-1} Q_t m^t + y_{j-1} m^j$ 。
- 终止： $y_{j-1} = 0$ 或达到精度界。

核心原理解析

- $Q_j = \lfloor y_j m \rfloor$ 将小数首位放大到个位摘取； $y_{j+1} = y_j m - Q_j$ 余下纯小数继续。
- 产出顺序为 $Q_{-1} \rightarrow Q_{-2} \dots$ （高位到低位），正序拼接。

第二章 数据系统与物理存储

本章自底向上探讨数据的底层表示，从比特流出发，逐步建立数的表示与字符的编码。

2.1 数据、数据元素与数据对象

数据结构研究的出发点，是把现实世界中的信息抽象为计算机能够处理的形式。为此，必须先把几个由粗到细、由整体到局部的术语厘清：**数据**、**数据对象**、**数据元素**与**数据项**。它们共同刻画了“信息”在不同粒度上的组织方式。

- **数据 (Data)**: 信息的载体，是对客观事物的符号表示。凡能输入计算机并被程序处理的符号集合都称为数据，记作 $\mathcal{D}$ 。整数、字符、图像、声音均可视为数据。
- **数据项 (Data Item)**: 数据不可分割的最小单位，也称字段 (field)，记作 f_j 。例如“学号”“姓名”各自就是一个数据项。
- **数据元素 (Data Element)**: 数据的基本单位，由若干数据项组成，也称记录 (record)，记作 e_i 。它是程序处理时的整体对象。
- **数据对象 (Data Object)**: 具有相同性质的数据元素的集合，记作 $\mathcal{O}$ 。它是某一数据元素的子集，是数据的子集。

数据元素与数据项之间的组成关系可形式化地写为

$$e_i = \langle f_{i1}, f_{i2}, \dots, f_{ik} \rangle, \quad f_{ij} \in \mathcal{F}$$

而数据对象则是满足共同性质 P 的元素之集：

$$\mathcal{O} = \{e \mid e \text{ 具有性质 } P\} \subseteq \mathcal{D}$$

四者的粒度由大到小依次为：数据 $\supseteq$ 数据对象 $\supseteq$ 数据元素 $\supseteq$ 数据项。下面用一个“学生表”把抽象概念落地。

术语	定义	学生表示例
数据项（字段）	不可分割的最小单位	学号、姓名、性别、年龄
数据元素（记录）	由若干数据项组成的基本单位	一行学生记录 (2021001, 张三, ...)
数据对象	相同性质数据元素的集合	全体学生构成的学生集合
数据	一切可被计算机处理的符号集合	整个数据库中的所有表

表 2.1: 数据、数据对象、数据元素、数据项的层次对比

示例: 设学生记录 $e_1 = \langle 2021001, \text{张三}, \text{男}, 19 \rangle$ ，则 2021001、张三、男、19 均为数据项， e_1 为一个数据元素，而所有学生记录 $\{e_1, e_2, \dots\}$ 构成一个数据对象。可见，同一个符号既可作数据项，也可在更高层次上组合为数据元素。

2.2 逻辑结构与存储结构的关系

仅知道数据元素本身还不够，真正决定数据结构性质的是元素之间的关系。由此引入两个既相互独立、又彼此制约的概念。

- **逻辑结构 (Logical Structure)**: 数据元素之间抽象关系的整体，是对问题的数学描述，与计算机无关。它只回答“谁与谁相邻、谁是谁的前驱”。
- **存储结构 (物理结构, Storage Structure)**: 逻辑结构在计算机存储器中的映像，即如何用存储单元与地址实现逻辑关系，是具体的、与机器相关的。

两者可分别形式化为二元组与三元组：

$$\text{逻辑结构} = (D, R), \quad \text{存储结构} = (D, R, M)$$

其中 D 为数据元素集合， R 为元素间关系的集合， M 为该关系在存储器中的映像方式（顺序、指针、索引、散列等）。二者的关系可概括为三点：

1. **逻辑结构是“设计”，存储结构是“实现”**。同一逻辑结构可有多种存储映像，例如线性表既可顺序存储，也可链式存储。
2. **逻辑结构决定操作语义，存储结构决定操作效率**。前者规定“插入”意味着什么，后者决定“插入”是 $O(1)$ 还是 $O(n)$ 。
3. **二者相互制约**。一旦选定存储结构，某些逻辑操作的实现便会受限，反之某些逻辑需求也会排除部分存储方案。

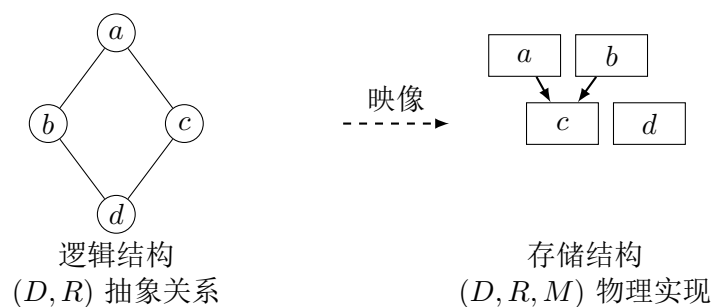


图 2.1: 同一逻辑结构到存储结构的映像

2.3 四种基本逻辑结构

按元素之间前驱/后继关系的数量，逻辑结构可分为四种基本形态：**集合**、**线性结构**、**树形结构**与**图形结构**。

- **集合**：元素之间除“同属一个集合”外别无其他关系，元素彼此平等、无序。
- **线性结构**：一对一关系。除首元无前驱、尾元无后继外，每个元素恰有一个直接前驱和一个直接后继（如线性表、栈、队列）。
- **树形结构**：一对多关系。每个元素至多一个直接前驱（双亲），可有多个直接后继（孩子）；无前驱者称为根（如二叉树、堆）。
- **图形结构**：多对多关系。每个元素的前驱与后继数目均不受限制，图中可含回路（如社交网络、交通网）。

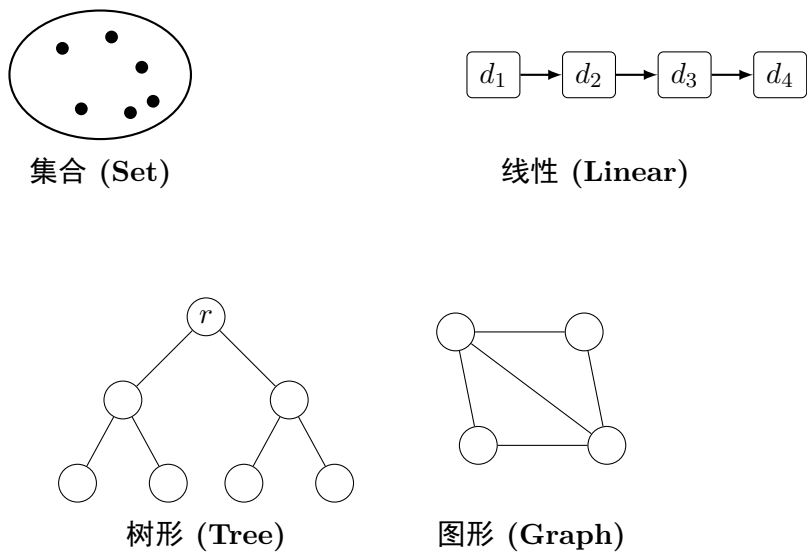


图 2.2: 四种基本逻辑结构

结构	前驱/后继	关系型	典型实例
集合	无	松散	并查集、哈希集合
线性结构	一对一	线性	线性表、栈、队列、串
树形结构	一对多	层次	二叉树、堆、B 树、字典树
图形结构	多对多	网状	有向/无向图、网络

表 2.2: 四种基本逻辑结构对比

2.4 四种存储映像

逻辑结构必须落到存储器上才能真正运行。常见的存储映像有四种：**顺序存储**、**链式存储**、**索引存储**与**散列存储**。它们各自用不同的手段实现“逻辑相邻”到“物理可达”的转换。

2.4.1 顺序存储 (Sequential Storage)

用一组**连续**的存储单元依次存放数据元素，使逻辑上相邻的元素在物理上也相邻。设每个元素占用 L 个存储单元，则第 i 个元素的地址为

$$LOC(a_i) = LOC(a_1) + (i - 1) \cdot L, \quad 1 \leq i \leq n$$

顺序存储无需附加指针，存储密度高，支持随机访问，这正是地址公式的直接结果。

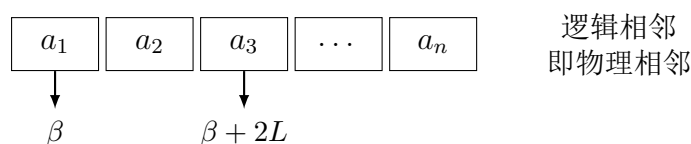


图 2.3: 顺序存储映像：逻辑相邻即物理相邻

2.4.2 链式存储 (Linked Storage)

每个数据元素与若干指针组成一个**结点**：数据域存放元素值，指针域存放后继（或前驱）的地址。逻辑相邻关系由指针给出，因此结点在物理上可以任意离散分布。

- **结点**： $n_k = (d_k, p_k)$ ，其中 p_k 指向下一结点。
- **不变式**： $p_k = \text{Addr}(n_{k+1})$ ，末尾结点 $p_{\text{last}} = \emptyset$ 。
- **代价**：随机访问退化为 $O(n)$ 的链式遍历，但插入、删除只需改写常数个指针即可完成。

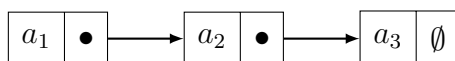


图 2.4: 链式存储映像：结点含指针，物理位置离散

2.4.3 索引存储 (Indexed Storage)

在存储数据元素的同时，另建一张**索引表**，表项形如 (k_i, p_i) ： k_i 为关键字， p_i 为该元素所在地址。查找时先在索引表中定位关键字，再按其地址取出数据元素。索引存储把“检索”与“存放”解耦，代价是额外的索引空间。

2.4.4 散列存储（Hash Storage）

通过散列函数 $h : K \rightarrow \{0, 1, \dots, m - 1\}$ 由关键字直接计算存储地址，理想情形下一次计算即可定位，期望查找时间为 $O(1)$ 。其代价是可能发生 **冲突**（ $k_1 \neq k_2$ 而 $h(k_1) = h(k_2)$ ），需借助开放寻址或链地址法加以解决（详见后续“哈希表”一节）。

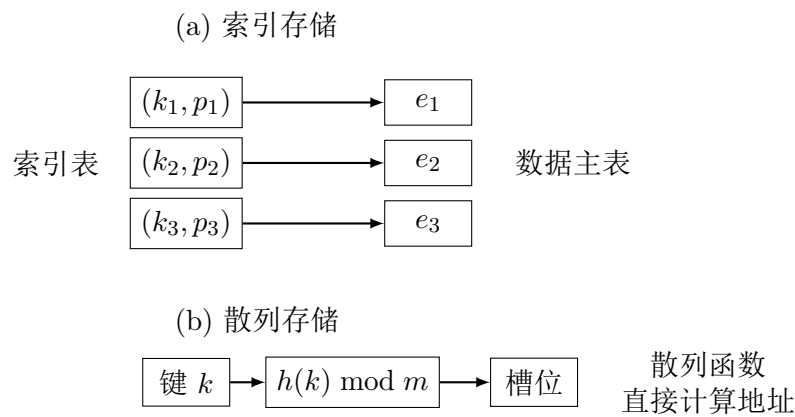


图 2.5: 索引存储与散列存储的基本思想

2.4.5 四种映像的对比

记存储密度 ρ （数据元素自身空间与结点总空间之比）：

$$\rho = \frac{\text{数据元素自身占用空间}}{\text{结点占用总空间}}, \quad 0 < \rho \leq 1$$

链式与索引存储因附加指针或表项而 $\rho < 1$ ，顺序存储通常 $\rho = 1$ 。

维度	顺序	链式	索引	散列
单元是否连续	连续	不要求	数据可离散	数据可离散
随机访问	$O(1)$	$O(n)$	$O(1)$ 经索引	期望 $O(1)$
插入/删除	$O(n)$	$O(1)$	$O(1)$ + 维护	期望 $O(1)$
额外空间	极少	指针	索引表	空槽/链
典型应用	数组、向量	链表	数据库索引、倒排	字典、集合

表 2.3: 四种存储映像的优缺点对比

2.5 顺序存储的地址计算

顺序存储的核心价值在于**随机访问**：只要知道公式，就能在 $O(1)$ 时间内算出任意元素的地址，无需从头遍历。

2.5.1 一维数组

设一维数组 $a_1, a_2, \dots, a_n$ 的起始地址为 $LOC(a_1)$ ，每个元素占 L 个单元，则下标从 1 起时

$$LOC(a_i) = LOC(a_1) + (i - 1) \cdot L, \quad 1 \leq i \leq n$$

若下标从 0 起，则相应改为 $LOC(a_i) = LOC(a_0) + i \cdot L$ 。

2.5.2 二维数组的行优先与列优先

设二维数组 A 有 m 行、 n 列，起始地址为 $LOC(a_{11})$ ，每元素占 L 个单元。**行优先** (Row-Major) 按行依次压平，元素 a_{ij} 之前的元素数为 $(i - 1)n + (j - 1)$ ：

$$LOC(a_{ij}) = LOC(a_{11}) + [(i - 1) \cdot n + (j - 1)] \cdot L$$

列优先 (Column-Major) 按列依次压平， a_{ij} 之前的元素数为 $(j - 1)m + (i - 1)$ ：

$$LOC(a_{ij}) = LOC(a_{11}) + [(j - 1) \cdot m + (i - 1)] \cdot L$$

可见两种布局的地址公式仅“行/列”与“ m/n ”互换。若下标从 0 起，行优先公式化为

$$LOC(a_{ij}) = LOC(a_{00}) + (i \cdot n + j) \cdot L$$

2.5.3 完整数值例题

题目：设二维数组 $A[1..4][1..5]$ ($m = 4$ 行， $n = 5$ 列)， $LOC(a_{11}) = 2000$ ，每个元素占 $L = 4$ 个字节。分别按行优先、列优先求 $LOC(a_{34})$ 与 $LOC(a_{25})$ 。

行优先求解：先算 a_{34} 之前的元素数

$$(3 - 1) \times 5 + (4 - 1) = 10 + 3 = 13$$

于是

$$LOC(a_{34}) = 2000 + 13 \times 4 = 2000 + 52 = 2052$$

再算 a_{25} ，其前元素数为 $(2 - 1) \times 5 + (5 - 1) = 5 + 4 = 9$ ，故

$$LOC(a_{25}) = 2000 + 9 \times 4 = 2000 + 36 = 2036$$

列优先求解： a_{34} 之前的元素数为

$$(4 - 1) \times 4 + (3 - 1) = 12 + 2 = 14$$

于是

$$LOC(a_{34}) = 2000 + 14 \times 4 = 2000 + 56 = 2056$$

再算 a_{25} ，其前元素数为 $(5 - 1) \times 4 + (2 - 1) = 16 + 1 = 17$ ，故

$$LOC(a_{25}) = 2000 + 17 \times 4 = 2000 + 68 = 2068$$

结论：同一元素在两种布局下的地址不同（2052 与 2056；2036 与 2068）。这说明地址计算必须与编译器的实际排列约定一致，否则会取到错误的数据。

2.6 索引存储与散列存储的基本思想

- **索引存储：**数据主表只负责存放元素，另设索引表 (k_i, p_i) 记录关键字到地址的映射。查找时先查索引表得到 p_i ，再访问主表取出元素。其本质是把“按关键字检索”转换为“按关键字查表 + 按地址取数”，用额外空间换取检索效率。
- **散列存储：**直接由关键字经散列函数计算存储地址， $h: K \rightarrow \{0, \dots, m-1\}$ ，使平均查找时间趋近常数。由于关键字空间通常远大于槽位数 m ，由鸽巢原理知冲突不可避免，须以探测或链地址等策略处理。

索引与散列代表了两种不同的设计哲学：索引是“以空间换时间，保序且可范围查询”，散列是“以计算换时间，均值极快但失去顺序”。二者常常并存于数据库与语言运行时之中。

2.7 抽象数据类型（ADT）

抽象数据类型（Abstract Data Type, ADT）是对一类数据的**数学抽象**，它只规定“有哪些数据、满足什么关系、可以施加哪些操作”，而**隐藏**这些操作在计算机内部的具体表示与实现。ADT 可形式化为三元组

$$ADT = (D, R, P)$$

其中 D 为数据对象， R 为 D 上的关系集合， P 为作用于 D 的基本操作集合。每个操作以“前置条件”与“后置条件”加以刻画，而不涉及实现细节。

ADT 具有两个层次：

1. **抽象层（对外接口）：**使用者的视角，只关心操作的语义与公理，不关心内部结构。
2. **实现层（对内表示）：**实现者的视角，选择顺序或链式等存储结构完成接口，并可更换而不影响使用者。

示例（ADT 复数）：以复数为例说明 ADT 的三要素。

- **数据对象 D ：** $D = \{(r, i) \mid r, i \in \mathbb{R}\}$ ，即所有复数。
- **数据关系 R ：**元素均为实数对的集合，形态固定为 (r, i) 结构。
- **基本操作 P ：**
 - $\text{Init}(c, r, i)$ ：构造实部为 r 、虚部为 i 的复数。

- $\text{Add}(c_1, c_2)$ 、 $\text{Sub}(c_1, c_2)$ 、 $\text{Mul}(c_1, c_2)$ ：复数四则运算。
- $\text{GetReal}(c)$ 、 $\text{GetImag}(c)$ ：读取实部、虚部，即观测算子。
- **实现与抽象分离**：同一个 ADT 复数既可用两个浮点数顺序存储，也可用极坐标表示；只要对外操作语义不变，使用者的代码无需改动。
- **与数据结构的区别**：ADT 是“接口/规约”，数据结构是“实现/载体”。前者回答“能做什么”，后者回答“怎么做到”。

2.8 本章小结与常见误区

本章沿“数据 $\rightarrow$ 逻辑结构 $\rightarrow$ 存储映像 $\rightarrow$ ADT”的主线梳理了数据结构的基本概念：数据由数据项组成数据元素，数据元素汇成数据对象；逻辑结构描述元素间的抽象关系，存储结构给出其物理实现；四种逻辑结构可映射为顺序、链式、索引、散列四种存储映像；顺序存储的地址公式实现了 $O(1)$ 随机访问，而 ADT 则把接口与实现彻底分离。

常见误区	正确理解
把“数据元素”等同于“数据项”	数据元素由若干数据项组成，粒度更粗
认为逻辑结构就是存储结构	逻辑结构抽象、与机器无关；存储结构是其物理映像，同一逻辑结构可有多种映像
认为顺序存储一定优于链式存储	顺序访问快但插入删除慢；链式反之，应按操作特征选择
把 ADT 等同于某个具体结构	ADT 是接口规约，可用顺序或链式等多方案实现
忽略索引/散列的额外空间代价	索引表与空槽同样占用空间，需在时间与空间之间权衡

表 2.4: 本章常见误区与辨析

易错提醒：二维数组地址计算时务必区分行优先与列优先，并确认下标起点是 0 还是 1；链式存储的“插入删除 $O(1)$ ”以已知目标位置为前提，若需先查找则仍为 $O(n)$ ；散列存储的平均 $O(1)$ 是期望时间，最坏情形仍可能退化为 $O(n)$ 。

第三章 线性结构与抽象数据类型

3.1 物理存储结构：线性表（Linear List）

线性表是有限序列 $L = \langle d_1, d_2, \dots, d_n \rangle$, $d_i \in \mathcal{D}$ 。

3.1.1 顺序存储结构（顺序表 / 数组）

- 对象：基址 β ，元大小 σ ，连续区间 $[\beta, \beta + n\sigma)$ 。
- 不变式（随机访问）： $\text{Addr}(i) = \beta + i\sigma$, $0 \leq i < n$ ，时间 $O(1)$ 。
- 多维压平（行优先）： $M \times N$ 矩阵逻辑 (i, j) 映为 $\text{Addr}(i, j) = \beta + (iN + j)\sigma$ ；列优先为对偶实例。

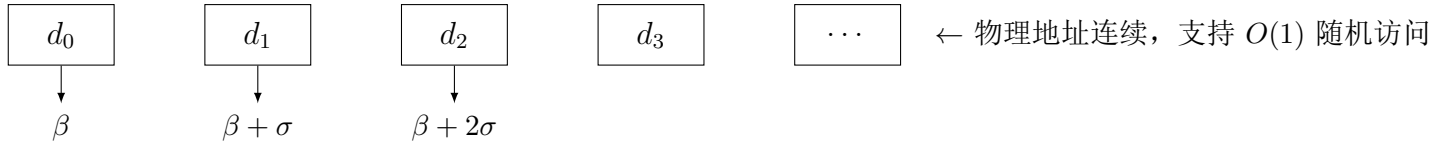


图 3.1: 一维顺序表物理内存布局

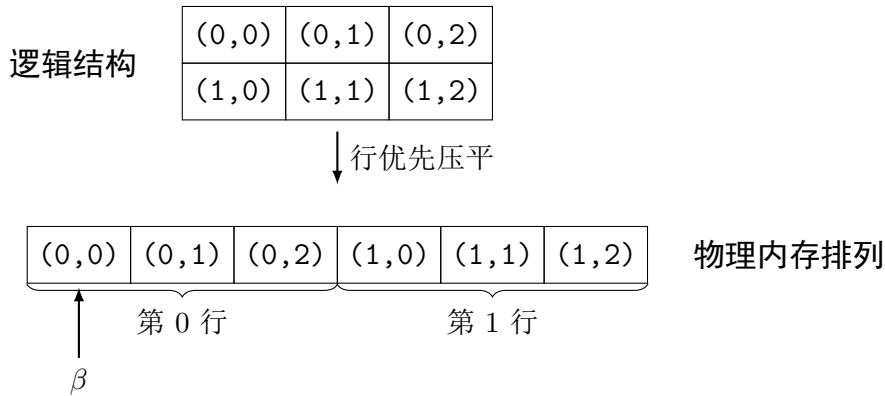


图 3.2: 二维数组行优先 (Row-Major) 压平排列

- 稀疏压缩: $A \in \mathbb{R}^{M \times N}$ ，三元组 $\mathcal{T} = \{(i, j, A_{ij}) \mid A_{ij} \neq 0\}$ ，空间 $O(|\mathcal{T}|)$ ，检索需索引结构。

3.1.2 链式存储结构（链表 / Link List）

- 对象：节点 $n_k = (d_k, p_k)$, $p_k \in \text{Memory} \cup \{\emptyset\}$ 。
- 不变式: $p_k = \text{Addr}(n_{k+1})$, $p_{\text{last}} = \emptyset$ ；双向加前驱，循环令 $p_{\text{last}} = \text{Addr}(n_1)$ 。
- 定理：随机访问 $O(n)$ ，头插删 $O(1)$ 。

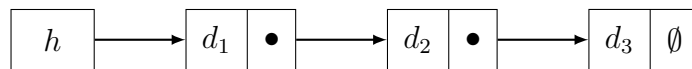


图 3.3: 单向链表 (Singly Linked List)

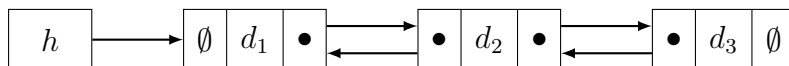


图 3.4: 双向链表 (Doubly Linked List)

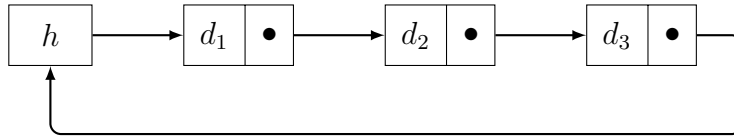


图 3.5: 循环链表 (Circular Linked List)

3.1.3 线性表操作规约

Algorithm 3 顺序表插入/删除规约

```
1: procedure SEQINSERT( $A, n, i, x$ )
2:   requires  $0 \leq i \leq n < m$ 
3:   for  $k \leftarrow n$  downto  $i + 1$  do  $A[k] \leftarrow A[k - 1]$ 
4:   end for
5:    $A[i] \leftarrow x$ 
6: end procedure
7: procedure SEQDELETE( $A, n, i$ )
8:   requires  $0 \leq i < n$ 
9:   for  $k \leftarrow i$  to  $n - 2$  do  $A[k] \leftarrow A[k + 1]$ 
10:  end for
11: end procedure
```

Algorithm 4 链式插入/删除规约 (单向)

```
1: procedure LINKINSERT( $prev, x$ )
2:    $node \leftarrow \text{new}(x)$ ;  $node.next \leftarrow prev.next$ ;  $prev.next \leftarrow node$ 
3: end procedure
4: procedure LINKDELETE( $prev$ )
5:    $prev.next \leftarrow prev.next.next$ 
6: end procedure
```

3.2 核心矛盾对比：顺序表 vs 链表

特性维度	顺序存储（数组）	链式存储（链表）
物理内存要求	连续区间 $[\beta, \beta + n\sigma)$	离散节点集合 $\{n_1, \dots, n_k\}$
元素访问	$O(1)$: $\text{Addr}(i) = \beta + i\sigma$	$O(n)$: 指针链遍历
插入/删除	$O(n)$: 平移 $d_{i+1} \dots d_n$	$O(1)$: 重写常数个指针（已知前驱）
空间代价	连续分配，扩容需 $O(n)$ 复制（摊还）	额外指针 $O(n\sigma_p)$ ，但无扩容复制

3.3 栈 (Stack)

3.3.1 代数定义

栈 S 是定义在元素集 E 上的代数结构，由以下算子与公理刻画：

- 构造算子：

- $\text{empty} : \rightarrow S$ （空栈）
- $\text{push} : S \times E \rightarrow S$ （压入）

- 观测算子：

- $\text{pop} : S \rightarrow S \cup \{\perp\}$ （弹出，偏函数）
- $\text{top} : S \rightarrow E \cup \{\perp\}$ （栈顶，偏函数）
- $\text{isEmpty} : S \rightarrow \mathbb{B}$ （判空）

公理系统 ($\forall s \in S, e \in E$):

1. $\text{isEmpty}(\text{empty}) = \text{true}$
2. $\text{isEmpty}(\text{push}(s, e)) = \text{false}$
3. $\text{pop}(\text{push}(s, e)) = s$
4. $\text{top}(\text{push}(s, e)) = e$
5. $\text{pop}(\text{empty}) = \perp, \text{top}(\text{empty}) = \perp$

3.3.2 物理实现方案

1. 顺序栈（有界实例）： S_m 映射到 $A[0..m-1]$, $t \in \{-1, \dots, m-1\}$, 不变式 $t < m-1$ 为 push 的定义域, $t \geq 0$ 为 pop 的定义域。
2. 链栈（无界实例）：映射到单向链表，表头即栈顶，无容量上限，为 S 的无界实例。

Algorithm 5 栈操作规约（有界顺序实例，数学层为偏函数）

- 1: 状态: $A[0..m-1], t \leftarrow -1$
 - 2: **procedure** $\text{PUSH}(e)$ $\triangleright$ requires $t < m-1$
 - 3: $t \leftarrow t + 1; A[t] \leftarrow e$
 - 4: **end procedure**
 - 5: **procedure** POP $\triangleright$ requires $t \geq 0$
 - 6: $e \leftarrow A[t]; t \leftarrow t - 1; \text{return } e$
 - 7: **end procedure**
-

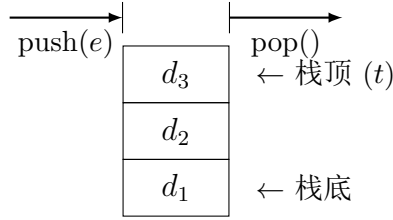


图 3.6: 栈 (Stack) 逻辑模型

3.4 队列 (Queue)

3.4.1 代数定义

队列 Q 是定义在 E 上的代数结构:

- 构造算子: $\text{emptyQ} : \rightarrow Q$, $\text{enqueue} : Q \times E \rightarrow Q$
- 观测算子: $\text{dequeue} : Q \rightarrow Q \cup \{\perp\}$, $\text{front} : Q \rightarrow E \cup \{\perp\}$, $\text{isEmptyQ} : Q \rightarrow \mathbb{B}$ (均为偏函数, 空队列上为 $\perp$)

公理系统 ($\forall q \in Q, e \in E$):

1. $\text{isEmptyQ}(\text{emptyQ}) = \text{true}$
2. $\text{dequeue}(\text{enqueue}(\text{emptyQ}, e)) = \text{emptyQ}$
3. $\text{dequeue}(\text{enqueue}(\text{enqueue}(q, e_1), e_2)) = \text{enqueue}(\text{dequeue}(\text{enqueue}(q, e_1)), e_2)$
4. $\text{front}(\text{enqueue}(\text{emptyQ}, e)) = e$
5. 若 $\text{isEmptyQ}(q) = \text{false}$, 则 $\text{front}(\text{enqueue}(q, e)) = \text{front}(q)$

3.4.2 循环队列 (Circular Queue)

为消除假溢出, 将数组索引映射到剩余类环 $\mathbb{Z}_m$ 。

- 对象: $Q[0..m-1]$, $f, r \in \mathbb{Z}_m$ 。
- 不变式: $f \leftarrow (f+1) \bmod m$, $r \leftarrow (r+1) \bmod m$; 队空 $r = f$, 队满 $(r+1) \bmod m = f$ (牺牲一槽位的不变式实例, 其他实例以计数器区分)。
- 定理: 环上 $\text{enqueue}/\text{dequeue}$ 均为 $O(1)$ 。

Algorithm 6 循环队列规约 (有界实例, 数学层为偏函数)

- 1: 状态: $Q[0..m-1]$, $f \leftarrow 0, r \leftarrow 0$
 - 2: **procedure** ENQUEUE(e) $\triangleright$ requires $(r+1) \bmod m \neq f$
 - 3: $Q[r] \leftarrow e$; $r \leftarrow (r+1) \bmod m$
 - 4: **end procedure**
 - 5: **procedure** DEQUEUE $\triangleright$ requires $r \neq f$
 - 6: $e \leftarrow Q[f]$; $f \leftarrow (f+1) \bmod m$; **return** e
 - 7: **end procedure**
-

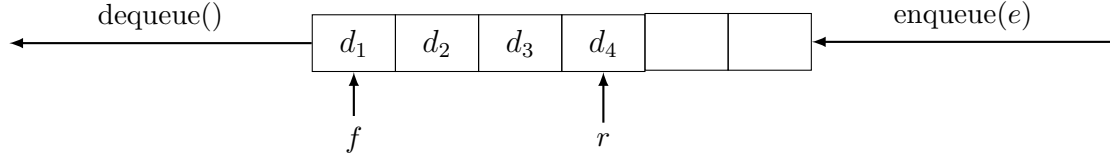


图 3.7: 普通队列 (Queue)

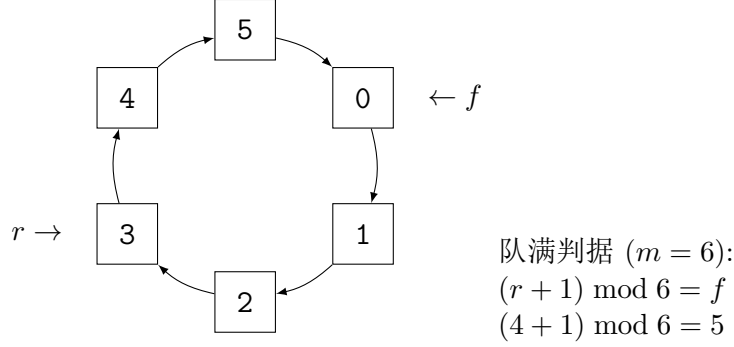


图 3.8: 循环队列 (Circular Queue)

3.5 双端队列 (Deque)

- 对象: 元素集 E 上的双端序列 D , 两端分别记 L, R 。
- 构造算子: $\text{emptyD} : \rightarrow D$, $\text{pushL}, \text{pushR} : D \times E \rightarrow D$ 。
- 观测算子: $\text{popL}, \text{popR} : D \rightarrow D \cup \{\perp\}$, $\text{frontL}, \text{frontR} : D \rightarrow E \cup \{\perp\}$, 均为偏函数。

公理: $\text{popL}(\text{pushL}(d, e)) = d$, $\text{popR}(\text{pushR}(d, e)) = d$, $\text{frontL}(\text{pushL}(d, e)) = e$, 空队操作得 $\perp$ 。栈与队列均为其限制: 仅用一端得栈, 两端异侧得队列。

Algorithm 7 双端队列规约 (循环数组实现)

```

1: 状态:  $A[0..m-1]$ ,  $f, r \in \mathbb{Z}_m$ ,  $sz$ 
2: procedure PUSH $L(e)$   $\triangleright$  requires  $sz < m$ 
3:    $f \leftarrow (f - 1 + m) \bmod m$ ;  $A[f] \leftarrow e$ ;  $sz \leftarrow sz + 1$ 
4: end procedure
5: procedure PUSH $R(e)$   $\triangleright$  requires  $sz < m$ 
6:    $A[r] \leftarrow e$ ;  $r \leftarrow (r + 1) \bmod m$ ;  $sz \leftarrow sz + 1$ 
7: end procedure
8: procedure POP $L$   $\triangleright$  requires  $sz > 0$ 
9:    $e \leftarrow A[f]$ ;  $f \leftarrow (f + 1) \bmod m$ ;  $sz \leftarrow sz - 1$ ; return  $e$ 
10: end procedure
11: procedure POP $R$   $\triangleright$  requires  $sz > 0$ 
12:    $r \leftarrow (r - 1 + m) \bmod m$ ;  $sz \leftarrow sz - 1$ ; return  $A[r]$ 
13: end procedure

```

3.6 字符串 (String)

3.6.1 形式化定义

- 对象：字母表 Σ 有限非空。
- 字符串幺半群： $\Sigma^* = \bigcup_{k=0}^{\infty} \Sigma^k$ ，空串 $\varepsilon \in \Sigma^0$ ，连接 $\circ : \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$ 结合， ε 为单位元。
- 不变式： $|s \circ t| = |s| + |t|$ ， $|\varepsilon| = 0$ 。
- 物理实例：底层以字符数组存储，边界以哨兵 $\# \notin \Sigma$ 界定。语言实例取 $\# = \backslash 0$ (C 字符串)，数学层仅以 $\# \notin \Sigma$ 表达；网页 `string-ops` 演示连接、子串等幺半群操作。



图 3.9: 字符串物理存储布局

3.6.2 串匹配算子 (KMP)

- 对象: 文本 $T[0..n-1]$, 模式 $P[0..m-1]$, $m \leq n$ 。
- 不变式 (前缀函数): $\pi[q] = \max\{k < m \mid k < q+1 \wedge P[0..k-1] = P[q-k+1..q]\}$, 即 P 前缀与后缀的最长重合。
- 定理: KMP 利用 π 跳过已匹配前缀, 时间 $O(n+m)$, 空间 $O(m)$, 最坏仍线性。

Algorithm 8 前缀函数 π 规约

```
1: procedure PREFIXFUNCTION( $P$ )
2:    $m \leftarrow |P|$ ;  $\pi[0] \leftarrow 0$ ;  $k \leftarrow 0$ 
3:   for  $q \leftarrow 1$  to  $m-1$  do
4:     while  $k > 0 \wedge P[k] \neq P[q]$  do  $k \leftarrow \pi[k-1]$ 
5:     end while
6:     if  $P[k] = P[q]$  then  $k \leftarrow k+1$ 
7:     end if
8:      $\pi[q] \leftarrow k$ 
9:   end for
10:  return  $\pi$ 
11: end procedure
```

Algorithm 9 KMP 匹配规约

```
1: procedure KMPMATCH( $T, P$ )
2:    $\pi \leftarrow \text{PrefixFunction}(P)$ ;  $q \leftarrow 0$ 
3:   for  $i \leftarrow 0$  to  $|T|-1$  do
4:     while  $q > 0 \wedge P[q] \neq T[i]$  do  $q \leftarrow \pi[q-1]$ 
5:     end while
6:     if  $P[q] = T[i]$  then  $q \leftarrow q+1$ 
7:     end if
8:     if  $q = m$  then return  $i-m+1$ 
9:     end if
10:  end for
11:  return  $-1$ 
12: end procedure
```

▷ 命中

3.6.3 BM 与 Rabin-Karp

- 对象：同 KMP 的 $T[0..n-1]$ 、 $P[0..m-1]$ 。
- **BM 不变式**：自右向左比对，坏字符 $bc[c] = \max\{k < m-1 \mid P[k] = c\}$ （不存在取 -1 ），好后缀按 P 内部重合右移；每轮跳跃 $\max(bc, gs)$ ，最优 $O(n/m)$ ，最坏 $O(nm)$ 。
- **RK 不变式**：滚动哈希 $H_{i+1} = (H_i - T[i]B^{m-1})B + T[i+m]$ ， $H_i = H(P) \implies$ 逐字验证；期望 $O(n+m)$ ，哈希碰撞时退化验证。

Algorithm 10 BM 坏字符规约

```
1: 预处理  $bc[c] \leftarrow P$  中  $c$  最右位置 ( $< m$ , 无则  $-1$ )
2: procedure BMMATCH( $T, P$ )
3:    $s \leftarrow 0$ 
4:   while  $s \leq n - m$  do
5:      $j \leftarrow m - 1$ 
6:     while  $j \geq 0 \wedge P[j] = T[s+j]$  do  $j \leftarrow j - 1$ 
7:     end while
8:     if  $j < 0$  then return  $s$ 
9:     end if
10:     $s \leftarrow s + \max(1, j - bc[T[s+j]])$ 
11:  end while
12:  return  $-1$ 
13: end procedure
```

Algorithm 11 Rabin-Karp 滚动哈希规约

```
1: procedure RKMATCH( $T, P, B, q$ )
2:    $hp \leftarrow H(P)$ ;  $ht \leftarrow H(T[0..m-1])$ ;  $h \leftarrow B^{m-1} \bmod q$ 
3:   for  $s \leftarrow 0$  to  $n - m$  do
4:     if  $ht = hp \wedge T[s..s+m-1] = P$  then return  $s$ 
5:     end if
6:     if  $s < n - m$  then  $ht \leftarrow ((ht - T[s]h)B + T[s+m]) \bmod q$ 
7:     end if
8:   end for
9:   return  $-1$ 
10: end procedure
```

3.7 高级映射与哈希表体系

3.7.1 字典 (Dictionary) / 映射 (Map)

- 对象：偏函数 $\mathcal{M}: K \rightarrow V$, $\text{dom}(\mathcal{M}) \subseteq K$ 。
- 不变式：单射于键 ($\mathcal{M}(k_1) = \mathcal{M}(k_2) \implies k_1 = k_2$ 在值可逆时，或键唯一性由表保证)。

3.7.2 哈希表 (Hash Table)

通过 $h: K \rightarrow \mathbb{Z}_m$ 将键空间压缩至槽位 $\{0, \dots, m-1\}$ 。

(1) 哈希函数

- 除留余数法： $h(k) = k \bmod m$, m 取质数以最小化冲突为实例化启发式。
- 理想哈希： $\forall k_i \neq k_j \implies h(k_i) \neq h(k_j)$ (鸽巢原理下仅当 $|K| \leq m$ 可实现)。

3.7.3 哈希碰撞 (Hash Collision)

(1) 碰撞必然性

由鸽巢原理，若 $|K| > m$ ，则 $\exists k_i \neq k_j$ 使 $h(k_i) = h(k_j)$ 。

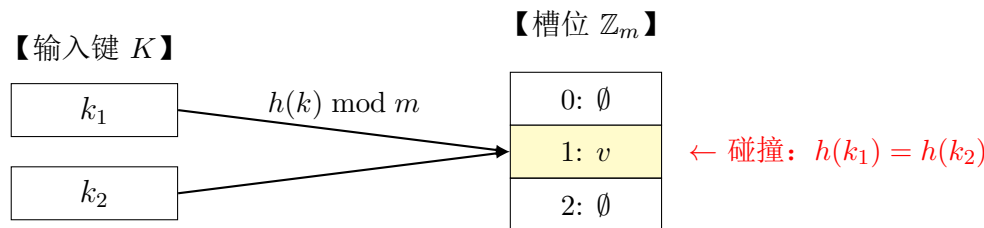


图 3.10: 哈希碰撞三大核心要素模型

(2) 冲突解决方案

1. 开放寻址法 (Open Addressing)

- 探测序列： $h_i(k) = (h(k) + \delta(i)) \bmod m$ ，所有槽位在主表中，冲突时按序列探测。
- 线性探测 $\delta(i) = i$: $h(k), h(k) + 1, \dots$ 期望 $O(1 + \alpha)$ ，但易主聚集。
- 二次探测 $\delta(i) = i^2$: $h(k), h(k) + 1, h(k) + 4, \dots$ 缓解主聚集； m 为质数且 $\alpha < 0.5$ 时可遍历。
- 双重散列 $h_i(k) = (h_1(k) + i \cdot h_2(k)) \bmod m$, $h_2(k) \not\equiv 0$ ，各键独立序列，最接近随机。
- 公共缺陷：删除需墓碑标记，再散列为扩容实例。

2. 链地址法 (Chaining)

- 槽位 $i \in \mathbb{Z}_m$ 维护 $L_i = \{(k, v) \mid h(k) = i\}$ 。
- 期望 $O(1 + \alpha)$, $\alpha = n/m$ 。
- 动态重构： $|L_i| > \tau$ 时重构为平衡树（实例取红黑树），最坏 $O(\log n)$ 。

Algorithm 12 哈希表查找与插入规约（以 h_i 泛化，线性为实例）

```
1: 状态:  $T[0..m-1]$ ,  $h_i(k)$ 
2: procedure SEARCH( $k$ )
3:    $i \leftarrow 0$ 
4:   repeat
5:      $j \leftarrow h_i(k)$ 
6:     if  $T[j] = \emptyset$  or  $T[j].key = k$  then return  $T[j]$ 
7:   end if
8:    $i \leftarrow i + 1$ 
9: until  $i = m$ 
10:  return  $\perp$ 
11: end procedure
12: procedure INSERT( $k, v$ )
13:    $item \leftarrow \text{Search}(k)$ 
14:   if  $item \neq \perp$  then  $item.value \leftarrow v$ 
15: else
16:    $i \leftarrow 0$ ;
17:   while True do  $j \leftarrow h_i(k)$ ;
18:     if  $T[j] = \emptyset$  then  $T[j] \leftarrow \langle k, v \rangle$ ; return
19:   end if;  $i \leftarrow i + 1$ 
20: end while
21: end if
22: end procedure
```

▷ 实例: $(h(k) + i) \bmod m$ 为线性

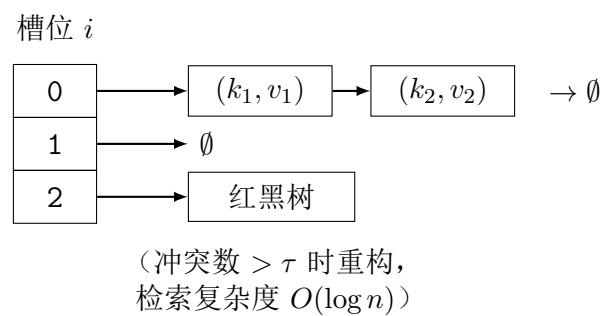


图 3.11: 链地址法与动态重构模型

3.8 字典树 (Trie) 与前缀检索

- **对象**: 字母表 Σ 上的字符串集 $W \subseteq \Sigma^*$ 。构造字符树: 根为空串, 每条边标记一个字符, 从根到某节点的路径即一个前缀。
- **不变式**: 节点子节点以字符为键唯一; 节点置词尾标记 $end(u)$, 当且仅当根到 u 的路径是一个完整词 (以此区分完整词与纯前缀)。
- **定理**: 插入/查找时间 $O(|w|)$, 与词表大小无关; 总空间 $O(\sum_{w \in W} |w|)$ 。前缀查询、最长公共前缀 (LCP) 天然高效, 是自动补全与拼写检查的基础。
- **基数树 (Radix / Patricia)**: 将无分支的单链路径压缩为一条边 (边标签为子串), 节点数降为 $O(\text{叶数})$, 读边标即可还原路径; 是 Trie 的空间优化变体。

Algorithm 13 字典树插入与查找规约

```
1: procedure TRIEINSERT( $root, w$ )
2:    $p \leftarrow root; i \leftarrow 0$ 
3:   while  $i < |w|$  do
4:     if  $w_i \notin p.children$  then  $p.children[w_i] \leftarrow node()$ 
5:     end if
6:      $p \leftarrow p.children[w_i]; i \leftarrow i + 1$ 
7:   end while
8:    $p.end \leftarrow \text{true}$ 
9: end procedure
10: procedure TRIESEARCH( $root, w$ )
11:    $p \leftarrow root$ 
12:   for  $c$  in  $w$  do
13:     if  $c \notin p.children$  then return false
14:     end if
15:      $p \leftarrow p.children[c]$ 
16:   end for
17:   return  $p.end$ 
18: end procedure
```

▷ 词尾才算命中 (前缀不算)

第四章 非线性结构 I：树与二叉树

树是一种一对多的非线性层次逻辑结构。在图论中，树被严密定义为：任意两个顶点间有且仅有一条路径的无环连通图。

4.1 通用树的基础属性

- 形式化定义：树 $T = (V, E)$ ，其中 V 为有限顶点集， $E \subseteq V \times V$ 为边集，满足 $|E| = |V| - 1$ 且连通。
- 节点分类：
 - 根节点 (Root)：唯一入度为 0 的节点 $r \in V$ 。
 - 叶子节点 (Leaf)：出度为 0 的节点集合 $L = \{v \in V \mid \text{out-degree}(v) = 0\}$ 。
 - 分支节点 (Internal)： $I = V \setminus (L \cup \{r\})$ 。
- 形态指标：
 - 度数 (Degree)： $\deg(v) = |\{u \mid (v, u) \in E\}|$ 。树的度 $D = \max_{v \in V} \deg(v)$ 。
 - 高度 (Height)： $H(T) = \max_{v \in V} \text{depth}(v)$ 。

4.2 二叉树 (Binary Tree) 及其形态特征

- 递归定义：二叉树 B 要么是空集 $\emptyset$ ，要么是一个三元组 $\langle d, L, R \rangle$ ，其中 d 为根节点数据， L 和 R 为互不相交的二叉树（左子树与右子树）。
- 节点约束： $\forall v \in V, \deg(v) \in \{0, 1, 2\}$ ，且严格区分左右子树。

根据形态与平衡度，二叉树有以下经典状态：

1. 偏斜树 (Skewed Tree)： $\forall v, \deg(v) \leq 1$ 。退化为线性结构，检索复杂度 $O(n)$ 。
2. 完全二叉树 (Complete Tree)：节点按层序编号 $1 \dots n$ ，与满二叉树前 n 个节点一一对应。
3. 满二叉树 (Full Tree)： $\forall v, \deg(v) \in \{0, 2\}$ ，且所有叶子节点深度相同。
4. 有序与平衡特化：二叉搜索树、AVL、红黑树、B/B+ 树均为在上述结构上附加序关系或平衡不变式的特化，详见本章后文各节（对象-不变式-定理-实例化）。

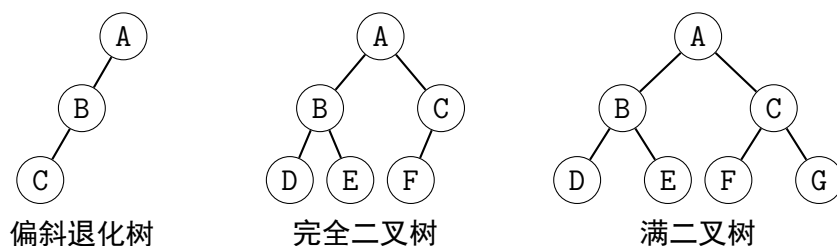


图 4.1: 二叉树核心形态对比

4.3 树的物理存储

树的存储分为数学映射与语言实例化两层。数学层只谈索引函数与元组结构；数组下标基址、指针结构体均为实例约定。

1. 顺序存储法（完全二叉树）

- 映射到数组 $A[0..n-1]$ 。
- 下标约定：本书取 0 基实例约定（C/Java 实例）。数学本质为完全二叉树的层序同构，1 基形式为 $l(i) = 2i, r(i) = 2i + 1$ ，与下式等价。
- 隐式纽带： $\forall i \in \{0, \dots, n-1\}$ ，
 - 左子节点索引： $l(i) = 2i + 1$ （若 $< n$ ）
 - 右子节点索引： $r(i) = 2i + 2$ （若 $< n$ ）
 - 父节点索引： $p(i) = \lfloor (i-1)/2 \rfloor$ （若 $i > 0$ ）

2. 链式存储法（二叉链表）

- 节点结构： $n = \langle lc, d, rc \rangle$ ，其中 $lc, rc \in \text{Node} \cup \{\emptyset\}$ 。
- 通用树转换定理（孩子兄弟表示法）：任意通用树同构于一棵二叉树。
- 节点结构 $n = \langle \text{firstChild}, d, \text{nextSibling} \rangle$ ，即该同构的语言实例化。

4.4 树的遍历 (Tree Traversal)

将非线性结构 T 映射为线性序列 $\sigma \in V^*$ 。

4.4.1 递归遍历 (Recursive Traversal)

Algorithm 14 二叉树递归遍历伪代码 (Binary Tree Traversal)

1: procedure PREORDER(u)	▷ 前序遍历
2: if $u = \emptyset$ then	
3: return	
4: end if	
5: Visit (u)	
6: Preorder(u .left)	
7: Preorder(u .right)	
8: end procedure	
9: procedure INORDER(u)	▷ 中序遍历
10: if $u = \emptyset$ then	
11: return	
12: end if	
13: Inorder(u .left)	
14: Visit (u)	
15: Inorder(u .right)	
16: end procedure	
17: procedure POSTORDER(u)	▷ 后序遍历
18: if $u = \emptyset$ then	
19: return	
20: end if	
21: Postorder(u .left)	
22: Postorder(u .right)	
23: Visit (u)	
24: end procedure	

4.4.2 非递归遍历 (Iterative Traversal)

递归即结构归纳，迭代即以显式栈 S 对调用栈的模拟（模拟引理），空间仍为 $O(H)$ ，而非消除复杂度。

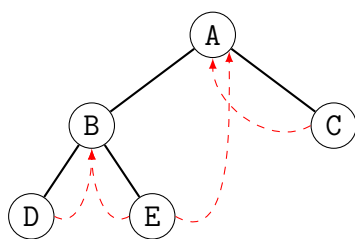
Algorithm 15 中序遍历迭代伪代码 (Iterative Inorder)

```
1: procedure INORDERITERATIVE( $root$ )  
2:    $S \leftarrow \emptyset$  ▷ 初始化空栈  
3:    $curr \leftarrow root$   
4:   while  $curr \neq \emptyset$  or  $S \neq \emptyset$  do  
5:     while  $curr \neq \emptyset$  do  
6:       Push( $S, curr$ )  
7:        $curr \leftarrow curr.left$   
8:     end while  
9:      $curr \leftarrow Pop(S)$   
10:    Visit( $curr$ )  
11:     $curr \leftarrow curr.right$   
12:  end while  
13: end procedure
```

4.4.3 线索二叉树 (Threaded Binary Tree)

为消除遍历时的栈空间开销 $O(H)$ ，利用空指针域存储遍历序中的前驱与后继。

- 数学定义：对于节点 u ，定义标记位 $LTag, RTag \in \{0, 1\}$ ：
 - 若 $u.left = \emptyset$ ，则 $u.left \leftarrow pre(u)$ （中序前驱），且 $LTag(u) = 1$ 。
 - 若 $u.right = \emptyset$ ，则 $u.right \leftarrow succ(u)$ （中序后继），且 $RTag(u) = 1$ 。
- 性质：线索化后，遍历操作退化为线性扫描，空间复杂度降至 $O(1)$ 。



虚线 = 线索指针；中序序列 $DBEAC$

图 4.2: 中序线索二叉树（红虚线指向中序前驱/后继）

4.4.4 遍历序列与树的唯一性

1. 唯一确定性定理:

- 前序 + 中序 $\implies$ 唯一确定二叉树。
- 后序 + 中序 $\implies$ 唯一确定二叉树。
- 前序 + 后序 $\implies$ 仅当树为满二叉树时唯一确定，否则存在歧义（无法区分左右子树边界）。

2. 构造算法:

Algorithm 16 由前序与中序序列构造二叉树

```
1: 输入: 前序序列  $P$ , 中序序列  $I$ 
2: procedure BUILDTREE( $P, I$ )
3:   if  $P = \emptyset$  or  $I = \emptyset$  then
4:     return  $\emptyset$ 
5:   end if
6:    $root \leftarrow P[0]$  ▷ 前序首元素为根
7:    $k \leftarrow \text{Index}(I, root)$  ▷ 在中序中定位根
8:    $I_L \leftarrow I[0..k-1], I_R \leftarrow I[k+1..|I|-1]$ 
9:    $P_L \leftarrow P[1..|I_L|], P_R \leftarrow P[|I_L|+1..|P|-1]$ 
10:   $root.left \leftarrow \text{BuildTree}(P_L, I_L)$ 
11:   $root.right \leftarrow \text{BuildTree}(P_R, I_R)$ 
12:  return  $root$ 
13: end procedure
```

4.5 二叉搜索树 (BST)

- **对象:** 二叉树 $B = \langle d, L, R \rangle$, 键来自全序集 $(K, <)$ 。
- **不变式 (序):** $\forall x \in L, x.val < d.val \wedge \forall y \in R, y.val > d.val$ 。等价地, 中序遍历序列 $\sigma_{in}(B)$ 严格递增。
- **定理:** 若不变式成立, 则检索、插入、删除沿一条根-叶路径进行, 时间 $O(H)$, 其中 H 为树高。偏斜时 $H = n - 1$, 退化为 $O(n)$; 期望随机插入下 $H = O(\log n)$ 。
- **操作规约:** $\text{Search}(x)$ 为序比较导航; $\text{Insert}(x)$ 为叶位扩展; $\text{Delete}(x)$ 为三态: 叶直接删、单支提升、双支以后继 $\min(R)$ 替代并递归删后继, 保持中序不变。

Algorithm 17 BST 规约 (Search/Insert/Delete)

```
1: procedure SEARCH( $root, x$ )
2:    $u \leftarrow root$ 
3:   while  $u \neq \text{nil} \wedge u.val \neq x$  do
4:     if  $x < u.val$  then  $u \leftarrow u.left$ 
5:     else  $u \leftarrow u.right$ 
6:   end if
7:   end while
8:   return  $u$ 
9: end procedure
10: procedure INSERT( $root, x$ )
11:   if  $root = \text{nil}$  then return  $\text{node}(x)$ 
12:   end if
13:   if  $x < root.val$  then  $root.left \leftarrow \text{Insert}(root.left, x)$ 
14:   else if  $x > root.val$  then  $root.right \leftarrow \text{Insert}(root.right, x)$ 
15:   end if
16:   return  $root$ 
17: end procedure
18: procedure DELETE( $root, x$ )
19:   if  $x < root.val$  then  $root.left \leftarrow \text{Delete}(root.left, x)$ 
20:   else if  $x > root.val$  then  $root.right \leftarrow \text{Delete}(root.right, x)$ 
21:   else if  $root.left = \text{nil}$  then return  $root.right$ 
22:   else if  $root.right = \text{nil}$  then return  $root.left$ 
23:   elses  $s \leftarrow \min(root.right)$ ;  $root.val \leftarrow s.val$ ;  $root.right \leftarrow \text{Delete}(root.right, s.val)$ 
24:   end if
25:   return  $root$ 
26: end procedure
```

4.6 平衡二叉树 (AVL)

- 对象: BST B , 记 $H(v)$ 为以 v 为根的子树高, 平衡因子 $bf(v) = H(L_v) - H(R_v)$ 。
- 不变式 (平衡): $\forall v \in V, |bf(v)| \leq 1$ 。
- 定理: AVL 树高 $H = O(\log n)$ (Fibonacci 下界: $n(H) \geq F_{H+3} - 1$)。插入/删除后沿回溯路径首个 $|bf| = 2$ 节点经一次旋转恢复全局不变式。
- 旋转同构: 左旋/右旋为保持中序序列不变的树同构。四型: LL (右旋)、RR (左旋)、LR (先左后右)、RL (先右后左)。旋转只重写常数条边, 为 $O(1)$ 。

Algorithm 18 AVL 插入与重平衡规约

```

1: procedure AVLINSERT( $v, x$ )
2:    $v \leftarrow$  BST-Insert( $v, x$ ); 更新  $H$  与  $bf$  回溯
3:   if  $|bf(v)| = 2$  then                                     ▷ 首个失衡点
4:     if  $bf(v) = 2 \wedge bf(v.left) \geq 0$  then  $v \leftarrow$  RightRotate( $v$ )           ▷ LL
5:     else if  $bf(v) = -2 \wedge bf(v.right) \leq 0$  then  $v \leftarrow$  LeftRotate( $v$ )       ▷ RR
6:     else if  $bf(v) = 2 \wedge bf(v.left) < 0$  then  $v.left \leftarrow$  LeftRotate( $v.left$ );  $v \leftarrow$  RightRotate( $v$ )   ▷ LR
7:     else  $v.right \leftarrow$  RightRotate( $v.right$ );  $v \leftarrow$  LeftRotate( $v$ )           ▷ RL
8:   end if
9: end if
10:  return  $v$ 
11: end procedure

```

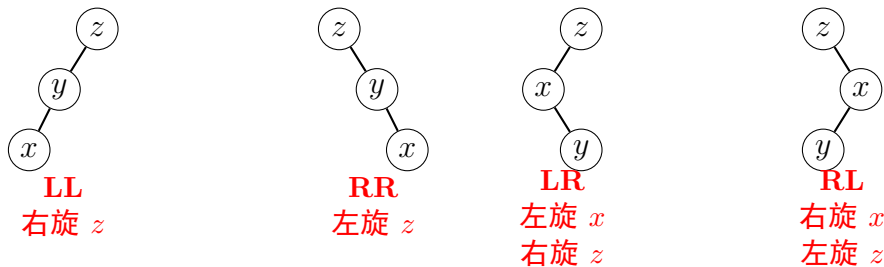


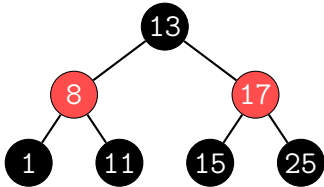
图 4.3: AVL 四种失衡形态与对应旋转 (z 为失衡点, y 孩子, x 孙)

4.7 红黑树 (Red-Black Tree)

- 对象: BST 附加颜色函数 $c: V \rightarrow \{\text{红}, \text{黑}\}$, 引入外部黑哨兵叶。
- 不变式: (1) 根为黑; (2) 红节点的子节点皆黑; (3) 任一根-叶路径黑高 bh 一致; (4) 新插入节点初染红。
- 定理 (黑高引理): $n \geq 2^{bh} - 1$, 故 $H \leq 2 \log_2(n+1) = O(\log n)$ 。插入修复至多两次旋转 + $O(\log n)$ 次变色; 删除修复为兄弟态讨论, 均保持黑高不变。
- 与 AVL 对比: AVL 平衡更紧 (检索更优), 红黑放宽到近似平衡 (插入/删除变色均摊更优), 同为 $O(\log n)$ 序结构。

Algorithm 19 红黑树插入修复规约

```
1:  $z \leftarrow \text{BST-Insert}(\text{root}, x)$ ;  $z.\text{color} \leftarrow \text{红}$ 
2: while  $z.\text{parent}.\text{color} = \text{红}$  do
3:   if  $z.\text{parent}$  为祖父左子 then
4:      $y \leftarrow$  祖父右子 ▷ 叔
5:     if  $y.\text{color} = \text{红}$  then  $y.\text{color} \leftarrow \text{黑}$ ;  $z.\text{parent}.\text{color} \leftarrow \text{黑}$ ;  $z.\text{parent}.\text{parent}.\text{color} \leftarrow \text{红}$ ;  $z \leftarrow z.\text{parent}.\text{parent}$ 
6:   else
7:     if  $z$  为右子 then  $z \leftarrow z.\text{parent}$ ; LeftRotate( $z$ )
8:     end if
9:      $z.\text{parent}.\text{color} \leftarrow \text{黑}$ ;  $z.\text{parent}.\text{parent}.\text{color} \leftarrow \text{红}$ ; RightRotate( $z.\text{parent}.\text{parent}$ )
10:  end if
11: else ▷ 镜像
12: end if
13: end while
14:  $\text{root}.\text{color} \leftarrow \text{黑}$ 
```



红节点子必黑；任一根-叶路径黑高 bh 一致

图 4.4: 红黑树着色示例（黑底为黑节点，红底为红节点）

4.8 B 树与 B+ 树 (B-Tree / B+ Tree)

- **对象**：多路搜索树，阶为 m （网页实例取 $m \in \{3, 4, 5\}$ ）。内节点存有序键与子树指针。
- **不变式 (B 树)**：除根外每节点键数 $k \in [\lceil m/2 \rceil - 1, m - 1]$ ，子数 $k + 1$ ；键有序分隔子树区间；所有叶等深。
- **定理**：高 $H = O(\log_m n)$ 。插入经分裂（中键上提）、删除经合并/借键，至多 $O(H)$ 个节点重写，分裂合并保持等深。
- **B+ 特化**：内节点仅存分隔键，数据全在叶层且叶链表相连；区间扫描为叶链表线性遍历，点查仍 $O(\log_m n)$ 。内节点分支因子更大，高度更低。

Algorithm 20 B 树插入分裂规约

```
1: procedure BINSERT(node, x)
2:   if node 为叶 then 插入 x 有序;
3:   else 选子 c 使区间含 x; BInsert(c, x)
4:   end if
5:   if |node.keys| = m then
6:     mid ← 中键; 分裂为 L, R 各  $\lceil m/2 \rceil - 1$  键
7:     if node 为根 then 新根  $\langle L, mid, R \rangle$ 
8:     else 将 mid 上提至父, 父接 L, R
9:     end if
10:  end if
11: end procedure
```

▷ 满

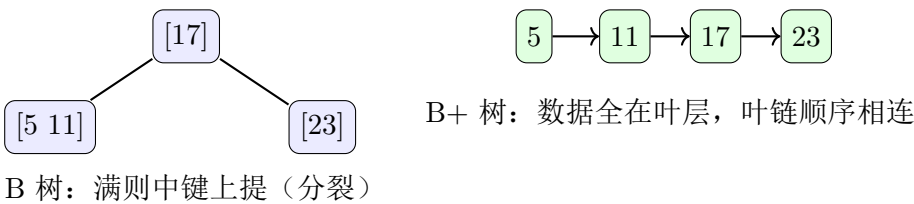


图 4.5: B 树节点分裂与 B+ 树叶链表

4.9 伸展树 (Splay Tree)

- **对象**: BST, 节点不含高度/颜色等平衡辅助字段; 仅在访问路径上做伸展旋转。
- **不变式 (自调整)**: 每次查找/插入/删除后, 将被访问节点 x 经 zig / zig-zig / zig-zag 旋转移到根; 结构随访问序列自适应。
- **定理 (摊还界)**: 取势能 $\Phi(T) = \sum_v \log_2 \text{size}(v)$ ($\text{size}(v)$ 为子树大小), 单次操作摊还代价 $O(\log n)$, m 次操作总 $O(m \log n)$; 单次最坏仍为 $O(n)$ 。
- **与 AVL / 红黑对比**: 无需存储平衡信息, 实现更简; 摊还意义下接近最优 (对非均匀访问序列优于严格平衡树), 但最坏单次 $O(n)$, 不适合硬实时场景。

Algorithm 21 伸展规约 (Splay)

```
1: while  $x.\text{parent} \neq \text{nil}$  do
2:    $p \leftarrow x.\text{parent}; g \leftarrow p.\text{parent}$ 
3:   if  $g = \text{nil}$  then
4:     单旋  $x$  至根                                ▷ zig
5:   else if  $x$  与  $p$  同侧 then
6:     先旋  $p$ , 再旋  $x$                                 ▷ zig-zig
7:   else
8:     先旋  $x$  (绕  $p$ ), 再旋  $x$  (绕  $g$ )                ▷ zig-zag
9:   end if
10: end while
11: return  $x$ 
```

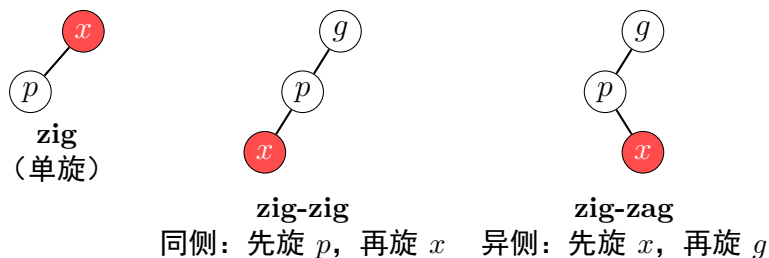


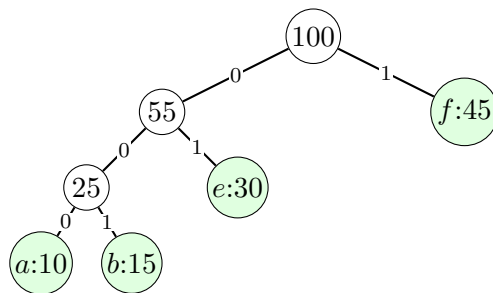
图 4.6: Splay 的三种旋转情形 (被访问节点 x 旋到根)

4.10 Huffman 最优树

- 对象：字母表 Σ ，频次 $freq: \Sigma \rightarrow \mathbb{N}$ ，带权路径 $WPL(T) = \sum_c freq(c) \cdot depth(c)$ 。
- 不变式（最优）：无前缀码，合并两最小频次节点得新子树，频次为和。
- 定理：Huffman 贪心构造的树 WPL 最小，时间 $O(n \log n)$ 。

Algorithm 22 Huffman 构造规约

```
1:  $Q \leftarrow$  按  $freq$  建最小堆，叶节点为  $(c, freq(c))$ 
2: for  $i \leftarrow 1$  to  $|\Sigma| - 1$  do
3:    $x \leftarrow \text{ExtractMin}(Q)$ ;  $y \leftarrow \text{ExtractMin}(Q)$ 
4:    $z \leftarrow \text{node}(freq \leftarrow freq(x) + freq(y), left \leftarrow x, right \leftarrow y)$ 
5:    $\text{Insert}(Q, z)$ 
6: end for
7: return  $\text{ExtractMin}(Q)$ 
```



边标 0/1 即前缀码； WPL 最小

图 4.7: Huffman 最优树（叶为「字符: 频次」，边标 0/1）

4.11 堆体系 (Heap)

- 定义：完全二叉树 T 满足堆序性质。
- 大顶堆 (Max Heap): $\forall v \in V, v.val \geq \text{leftChild}(v).val \wedge v.val \geq \text{rightChild}(v).val$ 。
- 小顶堆 (Min Heap): $\forall v \in V, v.val \leq \text{leftChild}(v).val \wedge v.val \leq \text{rightChild}(v).val$ 。
- 功能：提供 $O(1)$ 的最值访问与 $O(\log n)$ 的动态插入/删除，是优先队列的物理实现。
- 定理：自底向上建堆为 $O(n)$ （高度求和 $\sum H/2^H$ 收敛）；上滤/下滤保持堆序，优先队列 ExtractMin/DecreaseKey 为其实例化接口（供图论 Dijkstra 章复用）。

Algorithm 23 堆下滤操作伪代码 (Sift-Down for Max Heap)

```
1: 输入: 数组  $A$ , 节点索引  $i$ , 堆大小  $n$ 
2: procedure SIFTDOWN( $i, n$ )
3:    $largest \leftarrow i$ 
4:    $l \leftarrow 2i + 1$                                 ▷ 左孩子
5:    $r \leftarrow 2i + 2$                                 ▷ 右孩子
6:   if  $l < n$  and  $A[l] > A[largest]$  then
7:      $largest \leftarrow l$ 
8:   end if
9:   if  $r < n$  and  $A[r] > A[largest]$  then
10:     $largest \leftarrow r$ 
11:  end if
12:  if  $largest \neq i$  then
13:    Swap( $A[i], A[largest]$ )
14:    SiftDown( $largest, n$ )                                ▷ 递归下滤
15:  end if
16: end procedure
```

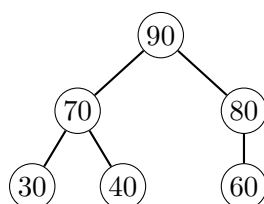
Algorithm 24 建堆规约 (BuildHeap)

```
1: procedure BUILDMAXHEAP( $A$ )
2:    $n \leftarrow |A|$ 
3:   for  $i \leftarrow \lfloor n/2 \rfloor - 1$  downto 0 do
4:     SiftDown( $A, i, n$ )
5:   end for
6: end procedure
```

- 定理：建堆时间为 $O(n)$ 。高度 h 层至多 $\lceil n/2^{h+1} \rceil$ 个节点下滤 $O(h)$ ，总和 $\sum_{h \geq 0} nh/2^{h+1} = O(n)$ 。

数组 A (层序存储)

90	70	80	30	40	60
0	1	2	3	4	5



大顶堆树形 ($l(i) = 2i + 1, r(i) = 2i + 2$)

图 4.8: 堆: 层序数组与完全二叉树一一对应

4.12 树上查询: LCA 倍增

- 对象: 有根树 T , $depth[v]$, $up[v][j] = v$ 的 2^j 祖先, $LOG = \lceil \log_2 |V| \rceil$ 。
- 不变式: $up[v][0] = parent(v)$, $up[v][j] = up[up[v][j-1]][j-1]$ 。
- 规约: 查询 (u, v) : (1) 提深者至同深; (2) 同步上跳至分叉点之子, 父即 LCA。预处理 $O(|V| \log |V|)$, 查询 $O(\log |V|)$ 。

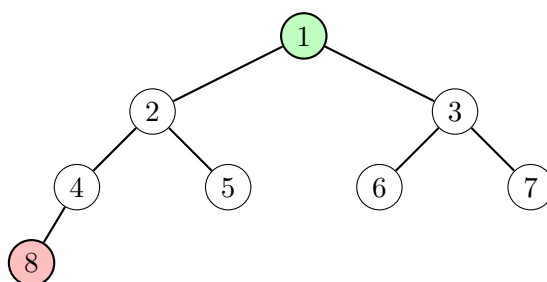
Algorithm 25 LCA 倍增规约

```

1: 预处理:  $depth[root] \leftarrow 0$ , DFS 求  $up[v][0], up[v][j] \leftarrow up[up[v][j-1]][j-1]$ 
2: procedure LCA( $u, v$ )
3:   if  $depth[u] < depth[v]$  then swap( $u, v$ )
4:   end if
5:   将  $u$  上提至  $depth[u] = depth[v]$ 
6:   if  $u = v$  then return  $u$ 
7:   end if
8:   for  $j \leftarrow LOG - 1$  downto 0 do
9:     if  $up[u][j] \neq up[v][j]$  then  $u \leftarrow up[u][j]; v \leftarrow up[v][j]$ 
10:    end if
11:  end for
12:  return  $parent[u]$ 
13: end procedure

```

▷ 二进制分解差值



绿 = LCA, 红 = u , 蓝 = v ; 先提 u 至同深, 再同步上跳

图 4.9: LCA 倍增: 同深对齐后二进制逐级上跳

第五章 非线性结构 II：图论

图是一种多对多的网状逻辑结构，用于建模现实世界中复杂的关联网络。

5.1 图的形式化定义

- 对象：图 $G = (V, E)$, V 有限, $E \subseteq V \times V$.
 - 无向图: E 为无序对 $\{\{u, v\}\}$; 有向图: E 为有序对 $\{(u, v)\}$ 。
- 扩展属性：权重 $w : E \rightarrow \mathbb{R}$; 邻接 $N : V \rightarrow \mathcal{P}(V)$, $N(u) = \{v \mid (u, v) \in E\}$ 。

5.2 边的拓扑分类

- 对象：有向图 G 。
- 不变式 (DAG): 不存在非空有向环 $\langle v_1, \dots, v_k \rangle$ 使 $(v_i, v_{i+1}) \in E$ 且 $v_1 = v_k$ 。
- 对象：无向图 G 。
- 不变式 (连通): $\forall u, v \in V$, 存在路径 $P = \langle u = x_0, \dots, x_k = v \rangle$ 使 $(x_i, x_{i+1}) \in E$ 。

5.3 连通性深度分析

- 强连通：有向图 $\forall u, v$ 存在 $u \rightarrow v$ 且 $v \rightarrow u$ 有向路径。
- 弱连通：忽略方向后的无向图连通。
- 桥： $e \in E$ 为桥 $\iff G' = (V, E \setminus \{e\})$ 的连通分量数严格增加。割点为点对偶。
- 定理：桥可由 DFS 的 DFN/LOW 在 $O(|V| + |E|)$ 内判定（见经典算法章 Tarjan）。

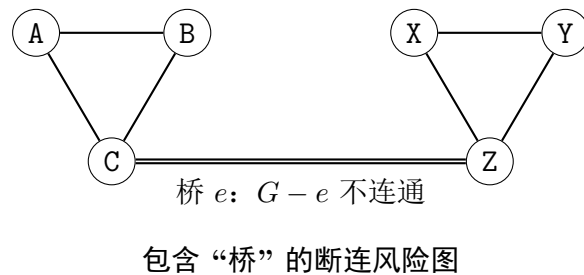
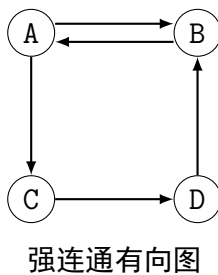


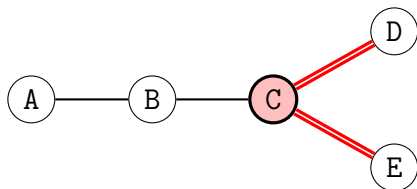
图 5.1: 图的位置连通性形态与“桥”

5.4 割点与双连通

- 对象: 无向图 G , DFS 树 $DFN[u], LOW[u] = \min\{DFN[u], \min_{v \text{ 为返祖边}} DFN[v], \min_{w \text{ 为子}} LOW[w]\}$ 。
- 不变式 (割点): (1) 根为割点 $\iff$ 子女数 ≥ 2 ; (2) 非根 u 为割点 $\iff$ 存在子女 v 使 $LOW[v] \geq DFN[u]$ 。
- 定理: DFS 一遍 $O(|V| + |E|)$ 可求全部割点/桥, 移除割点使连通分量增加。

Algorithm 26 割点判定规约 (Tarjan 扩展)

```
1:  $timer \leftarrow 0$ 
2: procedure DFS( $u, parent$ )
3:    $DFN[u] \leftarrow LOW[u] \leftarrow timer \leftarrow timer + 1; children \leftarrow 0$ 
4:   for all  $v \in N(u), v \neq parent$  do
5:     if  $DFN[v] = \text{nil}$  then
6:        $children \leftarrow children + 1; \text{DFS}(v, u); LOW[u] \leftarrow \min(LOW[u], LOW[v])$ 
7:       if  $parent = \text{nil} \wedge children \geq 2$  then  $u$  为割点
8:       end if
9:       if  $parent \neq \text{nil} \wedge LOW[v] \geq DFN[u]$  then  $u$  为割点
10:      end if
11:     else
12:        $LOW[u] \leftarrow \min(LOW[u], DFN[v])$ 
13:     end if
14:   end for
15: end procedure
```



红双线 = 桥; C 为割点: 移除后连通分量数增加

图 5.2: 割点与桥的判定 ($LOW[v] \geq DFN[u]$)

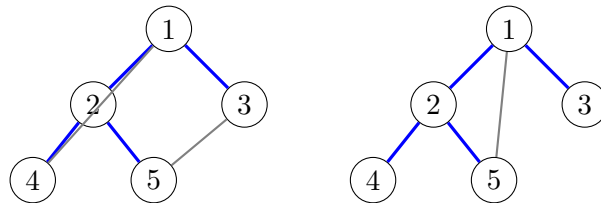
5.5 图的遍历与降维

5.5.1 遍历规约

- 对象: $\text{Visited} \subseteq V$, 输出序列 $\sigma \in V^*$ (数学层以序列代替回调 `Visit`)。
- 不变式 (BFS): 队列 F 为公平选择器, σ 按层扩展; (DFS): 以递归 (结构归纳) 扩展, σ 深度优先。
- 定理: 连通图上 BFS/DFS 均恰访每个可达顶点一次, 时间 $O(|V| + |E|)$ 。

Algorithm 27 图遍历规约 (以序列 σ 代 `Visit`)

```
1: 状态:  $\text{Visited} \leftarrow \emptyset, \sigma \leftarrow \langle \rangle, F \leftarrow \langle s \rangle$ 
2: procedure BFS( $s$ )
3:    $\text{Visited} \leftarrow \{s\}; F \leftarrow \langle s \rangle$ 
4:   while  $F \neq \langle \rangle$  do
5:      $u \leftarrow \text{popFront}(F); \sigma \leftarrow \sigma \circ \langle u \rangle$ 
6:     for all  $v \in N(u) \setminus \text{Visited}$  do  $\text{Visited} \leftarrow \text{Visited} \cup \{v\}; F \leftarrow F \circ \langle v \rangle$ 
7:   end for
8: end while
9: end procedure
10: procedure DFS( $u$ )
11:    $\text{Visited} \leftarrow \text{Visited} \cup \{u\}; \sigma \leftarrow \sigma \circ \langle u \rangle$ 
12:   for all  $v \in N(u) \setminus \text{Visited}$  do
13:     DFS( $v$ )
14:   end for
15: end procedure
```

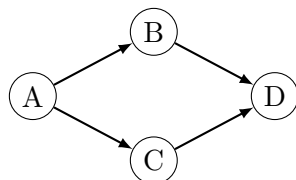


BFS 生成树 (队列按层) DFS 生成树 (递归/栈)

图 5.3: BFS 与 DFS 生成树 (蓝 = 树边, 灰 = 非树边)

5.5.2 拓扑排序

- 对象：DAG G 。
- 不变式：构造 $\sigma = \langle v_1, \dots, v_n \rangle$ 使 $\forall (u, v) \in E, u = v_i, v = v_j \implies i < j$ 。
- 定理： G 有拓扑序 $\iff G$ 为 DAG；Kahn 算法（反复取零入度集）与 DFS 逆后序均为构造性证明，时间 $O(|V| + |E|)$ 。



入度 $(A, B, C, D) = (0, 1, 1, 2)$
拓扑序： A, B, C, D 或 A, C, B, D

图 5.4: DAG 与拓扑序（Kahn：反复取零入度顶点）

5.6 图的物理存储实现

1. 邻接矩阵： $A \in \{0, 1\}^{n \times n}$, $A_{ij} = 1 \iff (v_i, v_j) \in E$ ；带权 $A_{ij} = w(v_i, v_j)$ 或 ∞ 。空间 $O(|V|^2)$ ，稠密图实例优。
2. 邻接表： $\text{Adj}[1..n]$, $\text{Adj}[i] = \{v \mid (v_i, v) \in E\}$ 以链表存储。空间 $O(|V| + |E|)$ ，稀疏图实例优。

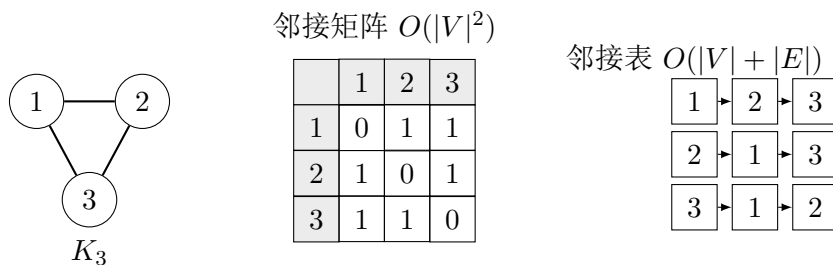


图 5.5: 图的两存储：邻接矩阵与邻接表

- 定理：两存储在同构意义下等价，仅时空权衡不同。

第六章 算法定义与渐近分析

本章探讨评价算法优劣的统一数学度量衡，建立时间与空间复杂度的渐进趋势评估模型，并给出分析算法复杂度的系统方法。

6.1 算法的定义与五个特性

6.1.1 从直观到形式化

直观定义：算法 (Algorithm) 是求解某一类问题的、由有限条指令组成的有穷操作序列。给出合法的输入，它按确定规则执行有限步后产生所要求的输出。

形式定义：设 $\mathcal{P}$ 为基本操作集，算法是 $\mathcal{P}$ 上的有限序列

$$A = \langle op_1, op_2, \dots, op_k \rangle, \quad op_i \in \mathcal{P}, \quad (6.1)$$

并配备三种控制构造子：顺序（依次执行）、分支（依条件选择）、循环（有条件重复）。其功能可用前后置断言（Hoare 三元组）刻画：

$$\{Pre\} A \{Post\}, \quad (6.2)$$

即只要前置条件 Pre 成立且算法终止，则后置条件 $Post$ 必成立。

6.1.2 五个基本特性

一个操作序列要成为算法，必须同时满足以下五个特性（见表 6.1）。

表 6.1: 算法的五个基本特性

特性	形式化描述	含义
有穷性	$\exists B, \text{步数}(A) \leq B$	对任意合法输入，算法在有限步内终止，且每一步在有限时间内完成
确定性	$\forall \text{状态}, \text{后继状态唯一}$	每条指令含义明确、无歧义，相同输入必得相同输出
可行性	$\forall op_i \in \mathcal{P}, op_i \text{ 可执行}$	每个操作原则上都可由基本运算有限次实现
输入	$I \in \mathcal{D}_{in}^*$	有零个或多个输入，取自特定对象的集合
输出	$O \in \mathcal{D}_{out}^+$	有一个或多个输出，且满足 $Post(I, O)$

6.1.3 算法与程序的区别

算法是与语言无关的求解方法，程序是算法在某种具体语言中的实现。二者关系见表 6.2。需要特别强调：程序不必满足有穷性（例如操作系统、服务器主循环可长期运行），而算法必须有穷。此外，同一个算法可以有多个程序实现，其执行时间仅相差常数因子（见第 6.4 节的分析法则）。

表 6.2: 算法与程序的对比

维度	算法	程序
存在形式	抽象的求解步骤（伪代码、数学记号）	具体语言的可执行代码
有穷性	必须终止	可不终止（交互式、系统程序）
确定性	每步唯一	可含随机、并发等非确定因素
依赖	与语言、机器无关	依赖编译器、运行环境
评价	渐进复杂度（ $n \rightarrow \infty$ 的趋势）	实际运行时间、内存占用

6.2 问题规模与基本操作

6.2.1 问题规模

问题规模 (Problem Size) 是刻画输入大小的量，记为 n 。它必须选取最能反映算法工作量的参数：排序问题中为待排元素个数；矩阵乘法中为矩阵阶数；图算法中常为顶点数 $|V|$ 与边数 $|E|$ ；求 $n!$ 时即为 n 本身。规模选错会导致复杂度阶被低估。

6.2.2 基本操作

基本操作 (Basic Operation) 是算法中执行次数最多、且其单次代价为常数 $O(1)$ 的操作。算法的运行时间可近似为

$$T(n) = c \cdot C(n), \quad (6.3)$$

其中 $C(n)$ 为基本操作执行次数， c 为一次基本操作的机器相关常数。常见基本操作：比较、赋值、加减乘除、数组下标访问、指针解引用。选择合适的基本操作，是把运行时间转化为可分析函数的关键。

6.2.3 最好、最坏与平均情况

同一算法在不同输入上代价不同，故需区分三种情形：

- **最好情况 (Best Case)**：使 $C(n)$ 最小的输入，给出下界直觉。如顺序查找首元素即命中。
- **最坏情况 (Worst Case)**：使 $C(n)$ 最大的输入，给出保证性上界，是最常用的度量。如顺序查找目标缺席或位于末尾。
- **平均情况 (Average Case)**：对输入按某概率分布取期望 $\mathbb{E}[C(n)]$ ，需先假设分布（常见为均匀分布）。

以顺序查找为例，设目标等概率出现在任意位置，平均比较次数为

$$\mathbb{E}[C(n)] = \frac{1}{n} \sum_{i=1}^n i = \frac{n+1}{2} = \Theta(n), \quad (6.4)$$

与最坏情况 $\Theta(n)$ 同阶。但快速排序的情形差异显著：最坏 $O(n^2)$ ，平均 $O(n \log n)$ ，见第 6.5 节。

6.3 渐进记号

渐进记号忽略常数因子与低阶项，只刻画 $n \rightarrow \infty$ 时增长的本质趋势。以下均设 $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$ 。

6.3.1 五种记号的形式化定义

- 大 O (渐进上界): $f(n) = O(g(n))$ 当且仅当

$$\exists c > 0, \exists n_0 > 0, \forall n \geq n_0: 0 \leq f(n) \leq c g(n). \quad (6.5)$$

- 大 Ω (渐进下界): $f(n) = \Omega(g(n))$ 当且仅当

$$\exists c > 0, \exists n_0 > 0, \forall n \geq n_0: 0 \leq c g(n) \leq f(n). \quad (6.6)$$

- 大 Θ (紧界): $f(n) = \Theta(g(n))$ 当且仅当 $f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$ ，即存在 $c_1, c_2 > 0, n_0$ 使 $c_1 g(n) \leq f(n) \leq c_2 g(n)$ 对一切 $n \geq n_0$ 成立。
- 小 o (严格上界): $f(n) = o(g(n))$ 当且仅当 $\forall c > 0, \exists n_0 > 0, \forall n \geq n_0: 0 \leq f(n) < c g(n)$ ，等价于 $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$ 。注意 O 允许同阶， o 不允许。
- 小 ω (严格下界): $f(n) = \omega(g(n))$ 当且仅当 $\forall c > 0, \exists n_0 > 0, \forall n \geq n_0: f(n) > c g(n)$ ，等价于 $\lim_{n \rightarrow \infty} f(n)/g(n) = +\infty$ 。

6.3.2 上界常数的几何意义

图 6.1 直观展示 $f(n) = O(g(n))$ 的含义：在 $n \geq n_0$ 的右半轴，曲线 $f(n)$ 始终位于 $c g(n)$ 之下，故 $c g(n)$ 是 $f(n)$ 的一个渐进上界。

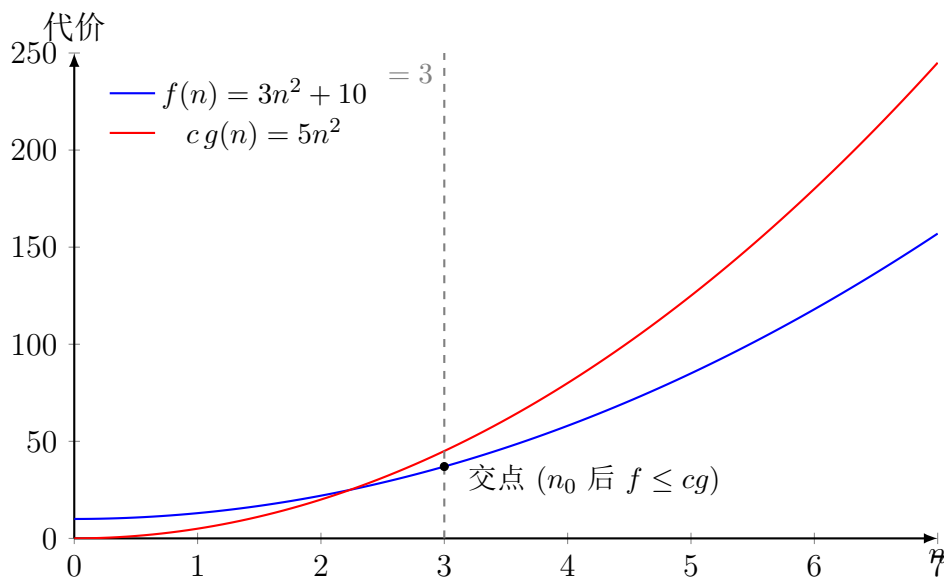


图 6.1: $f(n) = O(g(n))$ 的几何意义: $n \geq n_0$ 后 $f(n)$ 被 $c g(n)$ 压制

6.3.3 常见复杂度阶的排序

当 n 增大, 不同阶的函数增长速率急剧分化。常用阶由低到高的排序见表 6.3。

表 6.3: 常见复杂度阶由低到高排序

次序	阶	典型来源
1	$O(1)$	数组随机访问、哈希查找期望
2	$O(\log n)$	二分查找、平衡树检索
3	$O(n)$	线性扫描、计数排序
4	$O(n \log n)$	归并/堆排序、分治算法
5	$O(n^2)$	冒泡/插入排序、朴素矩阵乘法
6	$O(n^3)$	朴素矩阵乘法链、Floyd 全源最短路
7	$O(2^n)$	子集枚举、朴素斐波那契递归
8	$O(n!)$	全排列枚举、旅行商暴力解

阶的增长速率呈指数级分化（见图 6.2）：在 $n = 40$ 时 $O(2^n)$ 与 $O(n!)$ 已远超 $O(n^2)$ 数个数量级。这解释了为何降低复杂度阶（如用 $O(n \log n)$ 排序替代 $O(n^2)$ ）比优化常数因子重要得多。

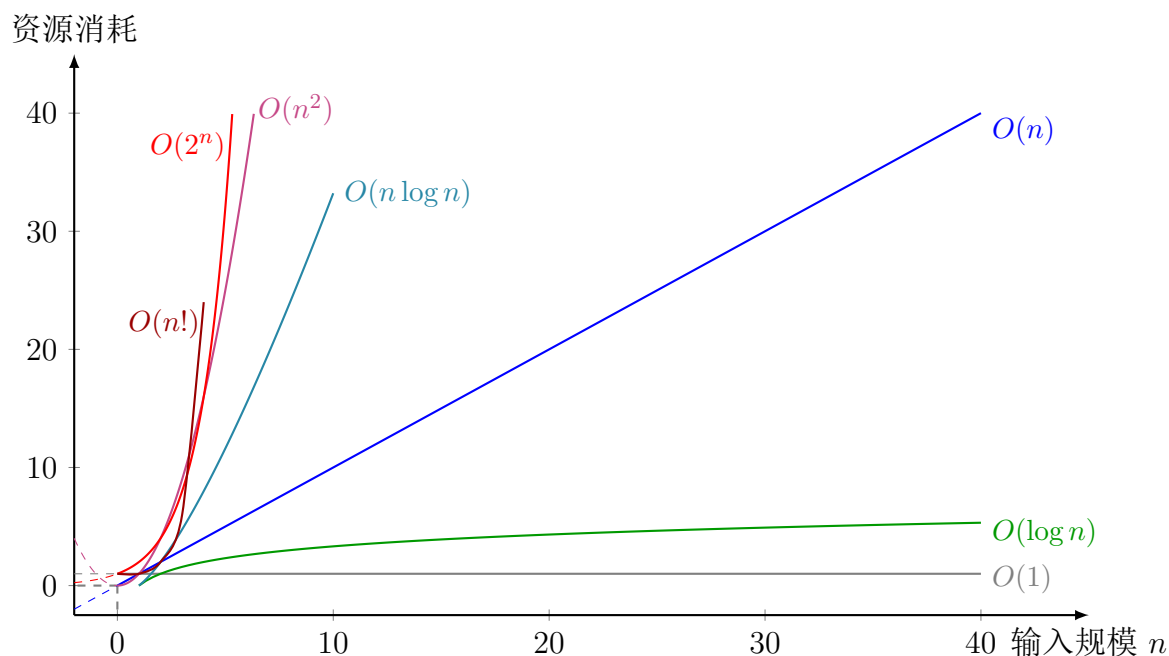


图 6.2: 常见复杂度阶的渐进曲线对比

6.4 算法分析的基本法则

6.4.1 加法法则与乘法法则

设 $T_1(n) = O(f(n))$, $T_2(n) = O(g(n))$ 。

- 加法法则（顺序/取最大）：两个先后执行的语句块总代价为

$$T(n) = T_1(n) + T_2(n) = O(f(n)) + O(g(n)) = O(\max(f(n), g(n))). \quad (6.7)$$

- 乘法法则（嵌套/取乘积）：若外层执行 $O(f(n))$ 次、每次触发代价 $O(g(n))$ 的内层，则总代价为

$$T(n) = T_1(n) \cdot T_2(n) = O(f(n)) \cdot O(g(n)) = O(f(n)g(n)). \quad (6.8)$$

加法法则的直觉是「木桶由最长板决定」；乘法法则对应循环嵌套。二者可反复套用，将复杂程序逐层化简。

6.4.2 代码片段复杂度例题

例 1（单层线性循环）：如下片段外层循环执行 n 次，循环体为 $O(1)$ 。由乘法法则 $T(n) = n \cdot O(1) = O(n)$ 。

Algorithm 28 单层循环求和

```
1:  $s \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:    $s \leftarrow s + A[i]$ 
4: end for
5: return  $s$ 
```

例 2（双重嵌套循环）：内外层各 n 次，循环体 $O(1)$ 。由乘法法则 $T(n) = O(n) \cdot O(n) = O(n^2)$ 。

Algorithm 29 双重嵌套循环

```
1:  $s \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:   for  $j \leftarrow 1$  to  $n$  do
4:      $s \leftarrow s + A[i] \cdot B[j]$ 
5:   end for
6: end for
```

例 3（对数循环）：变量 i 每次翻倍，迭代次数为满足 $2^k \leq n$ 的最大 k 加一，即 $\lfloor \log_2 n \rfloor + 1$ ，故 $T(n) = O(\log n)$ 。

Algorithm 30 倍增循环

```
1:  $i \leftarrow 1$ 
2: while  $i \leq n$  do
3:    $i \leftarrow 2i$ 
4: end while
```

例 4（外层线性、内层对数）：外层 n 次，内层倍增循环 $O(\log n)$ ，由乘法法则得 $T(n) = O(n \log n)$ 。

Algorithm 31 线性外层嵌套对数内层

```
1: for  $i \leftarrow 1$  to  $n$  do
2:    $j \leftarrow 1$ 
3:   while  $j \leq n$  do
4:      $j \leftarrow 2j$ 
5:   end while
6: end for
```

例 5 (条件递减): 如下片段中 i 每次减 2, 执行 $\lceil n/2 \rceil$ 次, $T(n) = O(n)$; 可见循环步长只改变常数因子, 不改变阶。

Algorithm 32 步长非一的线性循环

```
1:  $i \leftarrow n$ 
2: while  $i \geq 1$  do
3:    $s \leftarrow s + A[i]$ 
4:    $i \leftarrow i - 2$ 
5: end while
```

6.5 递归算法的复杂度分析

6.5.1 递推式与递归树

许多分治算法的代价满足**递推式 (Recurrence)**

$$T(n) = aT(n/b) + f(n), \quad (6.9)$$

其含义为: 把规模 n 的问题分解为 a 个规模 n/b 的子问题, 分解与合并的额外代价为 $f(n)$ 。 $a \geq 1, b > 1$ 。

递归树 (Recursion Tree) 把递推式逐层展开: 根为 $f(n)$; 第 j 层有 a^j 个节点, 每节点代价 $f(n/b^j)$, 故该层总代价为 $a^j f(n/b^j)$; 当 $n/b^L = 1$ 时到达叶层, 深度 $L = \log_b n$, 叶节点数 $a^L = n^{\log_b a}$, 每叶代价 $\Theta(1)$ 。总代价为各层之和。图 6.3 给出 $T(n) = 2T(n/2) + n$ 的递归树。

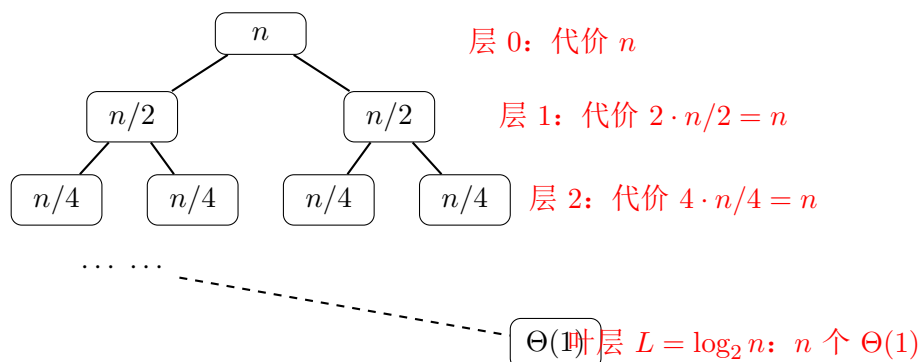


图 6.3: $T(n) = 2T(n/2) + n$ 的递归树: 每层代价均为 n

由图可见每层代价均为 n , 共 $L + 1 = \log_2 n + 1$ 层, 于是

$$T(n) = n(\log_2 n + 1) = \Theta(n \log n). \quad (6.10)$$

6.5.2 代入法

代入法 (Substitution Method): 先猜想解的界, 再用数学归纳法验证并求出常数。以 $T(n) = 2T(n/2) + n$ 为例, 猜想 $T(n) = O(n \log n)$, 即证 $T(n) \leq cn \log n$ 。

$$\begin{aligned} T(n) &= 2T(n/2) + n \\ &\leq 2c(n/2) \log_2(n/2) + n \\ &= cn(\log_2 n - 1) + n \\ &= cn \log_2 n - (c-1)n \\ &\leq cn \log_2 n, \quad \text{当 } c \geq 1. \end{aligned} \tag{6.11}$$

归纳成立, 故 $T(n) = O(n \log n)$; 再证下界 $T(n) = \Omega(n \log n)$ 即得 $\Theta(n \log n)$ 。代入法的要点是保留低阶项以凑出目标式子, 并在最后取定足够大的常数 c 。

6.5.3 主定理 (Master Theorem)

对形如 $T(n) = aT(n/b) + f(n)$ ($a \geq 1, b > 1$) 的递推式, 令临界值 $n^{\log_b a}$, 比较它与 $f(n)$ 的阶:

表 6.4: 主定理的三种情形 ($\varepsilon > 0$ 为任意小常数)

情形	条件	结论
1	$f(n) = O(n^{\log_b a - \varepsilon})$	$T(n) = \Theta(n^{\log_b a})$ (叶层主导)
2	$f(n) = \Theta(n^{\log_b a})$	$T(n) = \Theta(n^{\log_b a} \log n)$ (各层均衡)
3	$f(n) = \Omega(n^{\log_b a + \varepsilon})$ 且 $af(n/b) \leq cf(n)$ ($c < 1$)	$T(n) = \Theta(f(n))$ (根层主导)

情形 3 中的附加条件 $af(n/b) \leq cf(n)$ ($c < 1$) 称为正则条件, 用于保证子问题代价随层数指数衰减。

6.5.4 主定理例题

例 6: $T(n) = 2T(n/2) + n$ 。此处 $a = 2, b = 2$, 临界值 $n^{\log_2 2} = n$ 。而 $f(n) = n = \Theta(n)$, 落入情形 2, 故

$$T(n) = \Theta(n \log n), \tag{6.12}$$

与递归树、代入法的结论一致, 对应归并排序的复杂度。

例 7: $T(n) = 2T(n/2) + n^2$ 。仍为 $a = 2, b = 2$, 临界值 n , 而 $f(n) = n^2 = \Omega(n^{1+\varepsilon})$ (取 $\varepsilon = 1$)。检查正则条件: $af(n/b) = 2 \cdot (n/2)^2 = n^2/2 \leq cn^2$ (取 $c = 1/2 < 1$) 成立, 落入情形 3, 故

$$T(n) = \Theta(n^2). \tag{6.13}$$

例 8: $T(n) = 9T(n/3) + n$ 。临界值 $n^{\log_3 9} = n^2$, 而 $f(n) = n = O(n^{2-\varepsilon})$, 落入情形 1, 故 $T(n) = \Theta(n^2)$ 。

6.5.5 归并排序的算法与复杂度

归并排序是上述递推式的典型来源：将数组二分，递归排序左右两半，再在线性时间内归并。

Algorithm 33 归并排序 (Merge Sort)

```
1: procedure MERGESORT( $A, lo, hi$ )
2:   if  $lo \geq hi$  then
3:     return
4:   end if
5:    $mid \leftarrow \lfloor (lo + hi) / 2 \rfloor$ 
6:   MergeSort( $A, lo, mid$ )
7:   MergeSort( $A, mid + 1, hi$ )
8:   Merge( $A, lo, mid, hi$ )
9: end procedure
10: procedure MERGE( $A, lo, mid, hi$ )
11:   将  $A[lo..mid]$  与  $A[mid + 1..hi]$  按序并入临时数组  $B$ 
12:   将  $B$  复制回  $A[lo..hi]$ 
13: end procedure
```

▷ 至多一个元素已有序
▷ 递归左半
▷ 递归右半
▷ 线性合并

设归并代价为 cn ，递归代价满足 $T(n) = 2T(n/2) + \Theta(n)$ ，由主定理情形 2 得 $T(n) = \Theta(n \log n)$ ；额外空间为临时数组 B ，为 $\Theta(n)$ （见第 6.6 节）。

6.6 空间复杂度

空间复杂度 (Space Complexity) 度量算法运行所需存储随 n 的增长，通常只计辅助空间（不含输入本身）。它由三部分构成：

- 原地 (In-place)：仅使用常数个辅助单元， $S(n) = O(1)$ 。如冒泡排序、堆排序、数组逆置（借助常数槽位 τ 交换）。
- 递归栈 (Recursion Stack)：递归深度为 H 时，调用栈所占辅助空间为 $O(H)$ 。平衡分治（如归并、快排期望）为 $O(\log n)$ ；偏斜递归（如朴素斐波那契、退化的快排）为 $O(n)$ 。
- 额外数组/结构：显式分配与 n 成比例的空间。归并排序需 $\Theta(n)$ 临时数组；哈希表、邻接矩阵为 $O(n)$ 、 $O(n^2)$ 。

表 6.5 汇总常见算法的空间代价。

表 6.5: 常见算法的空间复杂度对比

算法	辅助空间	说明
数组逆置	$O(1)$	常数槽位交换，原地
冒泡 / 插入 / 选择排序	$O(1)$	原地排序
堆排序	$O(1)$	原地，递归下滤可改为迭代
归并排序	$O(n)$	需等大临时数组
快速排序（期望）	$O(\log n)$	递归栈深度期望 $O(\log n)$
快速排序（最坏）	$O(n)$	划分极度不平衡
朴素斐波那契递归	$O(n)$	递归深度 n

需要区分「输入空间」与「辅助空间」。归并排序总占用 $O(n) + O(n) = O(n)$ ，其额外部分为线性；而原地排序的额外部分为常数。分析时要明确报告哪一部分，否则结论会不一致。

6.7 本章小结与常见误区

6.7.1 小结

- 算法是满足有穷性、确定性、可行性、输入、输出五特性的有限操作序列，与具体程序实现相区别。
- 分析以问题规模 n 与基本操作为基础，须区分最好、最坏、平均三种情形；最坏情况给出保证性上界。
- 渐进记号 $O, \Omega, \Theta, o, \omega$ 用 $\exists c, n_0$ 形式化定义，忽略常数因子与低阶项。阶的排序为 $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$ 。
- 基本法则是加法法则 $O(\max)$ 与乘法法则 $O(\text{乘积})$ ，可逐层化简循环与嵌套。
- 递归分析有三件工具：递归树（看层求和）、代入法（归纳验证）、主定理（查表判定），三者结论应一致。
- 空间复杂度关注辅助空间：原地为 $O(1)$ ，递归栈为 $O(H)$ ，显式结构可能为 $O(n)$ 或更高。

6.7.2 常见误区

1. 把 O 当作「精确等于」： $T(n) = O(n)$ 只是上界，也可能是 $\Theta(\log n)$ 。写 $T(n) = O(n^2)$ 却实际为 $\Theta(n \log n)$ 虽正确但不紧。
2. 混淆 O 与 Θ ：断言「快排是 $O(n \log n)$ 」不严谨，因其最坏为 $O(n^2)$ ；准确说法为「快排期望/平均 $\Theta(n \log n)$ ，最坏 $\Theta(n^2)$ 」。
3. 忽略问题规模的定义：同一循环写法在不同规模参数下阶不同。图算法用 $|V|, |E|$ 双参数时，不可简单记作 $O(n)$ 。
4. 循环变量步长判断失误：「 $i *= 2$ 」是 $O(\log n)$ ，而「 $i += 2$ 」仍是 $O(n)$ ；只数「循环写了几个」而不数迭代次数会算错。
5. 主定理越界使用：情形 3 忘记校验正则条件，或 $f(n)$ 与临界值之间存在间隙（如 $f(n) = n \log n$ 与 $n^{\log_b a} = n$ 之间）时主定理不直接适用，需用递归树或代入法。
6. 遗漏递归栈空间：递归算法即使没有显式数组，也占 $O(H)$ 栈空间，勿误判为 $O(1)$ 。
7. 以常数优化替代降阶：把 $O(n^2)$ 的常数从 10 降到 1 远不如将算法改进为 $O(n \log n)$ ；当 n 很大时，阶的差异最终压倒一切常数因子。

第七章 动态经典算法与核心算子

本章探讨从静态结构向动态算子的演进，涵盖算法设计范式、排序、查找以及高级图论算法。

7.1 顶层设计范式

7.1.1 分治策略 (Divide and Conquer)

- 对象：规模 n 的问题 P 划分为 a 个规模 n/b 的子问题。
- 递推： $T(n) = aT(n/b) + f(n)$ ，由主定理决定渐进阶。

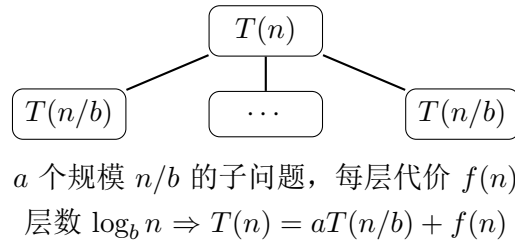


图 7.1: 分治递归树（主定理的直观解释）

7.1.2 贪心算法 (Greedy)

每步取局部最优 $s_i = \arg \max_{x \in C} \mathcal{F}(x)$ 。全局最优需：

1. 最优子结构： P 的最优解含子问题 P' 的最优解。
2. 贪心选择性：局部最优序列可拼接为全局最优。

7.1.3 动态规划 (Dynamic Programming)

- 对象：问题 P 具最优子结构与重叠子问题，状态 dp 递推。
- 不变式： dp 按拓扑序填表，已算子问题 $dp[s']$ 不重复计算。
- 定理：填表顺序保证 $dp[P]$ 即最优，时间常为状态数 $\times$ 转移代价。

Algorithm 34 01 背包 DP 规约

```
1:  $dp[0..n][0..W] \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:   for  $w \leftarrow 0$  to  $W$  do
4:     if  $wt[i] > w$  then  $dp[i][w] \leftarrow dp[i-1][w]$ 
5:     else  $dp[i][w] \leftarrow \max(dp[i-1][w], dp[i-1][w - wt[i]] + val[i])$ 
6:     end if
7:   end for
8: end for
9: return  $dp[n][W]$ 
```

Algorithm 35 LCS DP 规约

```
1:  $c[0..m][0..n] \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $m$  do
3:   for  $j \leftarrow 1$  to  $n$  do
4:     if  $X[i] = Y[j]$  then  $c[i][j] \leftarrow c[i-1][j-1] + 1$ 
5:     else if  $c[i-1][j] \geq c[i][j-1]$  then  $c[i][j] \leftarrow c[i-1][j]$ 
6:     else  $c[i][j] \leftarrow c[i][j-1]$ 
7:     end if
8:   end for
9: end for
10: return  $c[m][n]$ 
```

	B	D	C
A	0	0	0
B	1	1	1
C	1	1	2

对角线 = 字符匹配 (+1)，否则取上/左最大；回溯路径即 LCS

图 7.2: LCS 动态规划填表与回溯路径 ($X = ABC$, $Y = BDC$, $\text{LCS} = BC$)

7.2 排序算法算子 (Sort)

输入 $A = \langle a_1, \dots, a_n \rangle$, 目标 $A' = \langle a'_1, \dots, a'_n \rangle$ 使 $a'_1 \leq \dots \leq a'_n$ 。

7.2.1 比较类排序

1. 冒泡排序: 相邻交换消逆序对 $I(A) = |\{(i, j) \mid i < j \wedge a_i > a_j\}|$ 。
2. 选择排序: $a'_k = \min\{a_k, \dots, a_n\}$, 比较次数固定 $n(n-1)/2$ 。
3. 插入排序: 维护已序前缀 $A[1..k]$, 插入 a_{k+1} ; 已序时 $O(n)$ 。
4. 快速排序: 基准 p 划分 $A_L = \{x \leq p\}, A_R = \{x > p\}$, 递归。
5. 归并排序: $T(n) = 2T(n/2) + O(n)$, 稳定 $O(n \log n)$, 需 $O(n)$ 辅助。
6. 堆排序: 建堆 $O(n)$, 反复取根下滤, $O(n \log n)$, $O(1)$ 辅助。

Algorithm 36 快速排序规约

```
1: procedure QUICKSORT( $A, l, r$ )
2:   if  $l \geq r$  then return
3:   end if
4:    $p \leftarrow \text{Partition}(A, l, r)$ 
5:   QuickSort( $A, l, p-1$ ); QuickSort( $A, p+1, r$ )
6: end procedure
7: procedure PARTITION( $A, l, r$ )
8:    $pivot \leftarrow A[r]; i \leftarrow l-1$ 
9:   for  $j \leftarrow l$  to  $r-1$  do
10:    if  $A[j] \leq pivot$  then  $i \leftarrow i+1$ ; Swap( $A[i], A[j]$ )
11:    end if
12:  end for
13:  Swap( $A[i+1], A[r]$ ); return  $i+1$ 
14: end procedure
```

原始 A (末元素 6 为基准)

3	7	1	5	9	2	8	6
---	---	---	---	---	---	---	---

3	1	5	2	6	7	9	8
---	---	---	---	---	---	---	---

绿 $\{x \leq p\}$ | 红 p | 蓝 $\{x > p\}$

图 7.3: 快速排序的基准划分 (Partition)

7.2.2 非比较类排序

1. 计数排序: $C[x] = |\{a_i \mid a_i = x\}|$, 前缀 $S[x] = \sum_{k \leq x} C[k]$ 定位置, $O(n + k)$ 。
2. 桶排序: 值域 $[Min, Max]$ 分 k 区间 $B_1 \dots B_k$, 桶内排序后串联。
3. 基数排序: 按位 $d = 0 \dots D - 1$ 稳定排序 (以计数为子程序)。

Algorithm 37 归并排序规约

```
1: procedure MERGESORT( $A, l, r$ )
2:   if  $l \geq r$  then return
3:   end if
4:    $m \leftarrow \lfloor (l + r) / 2 \rfloor$ ; MERGESORT( $A, l, m$ ); MERGESORT( $A, m + 1, r$ ); MERGE( $A, l, m, r$ )
5: end procedure
6: procedure MERGE( $A, l, m, r$ )
7:    $B \leftarrow A[l..r]$ ;  $i \leftarrow l$ ;  $j \leftarrow m + 1$ ;  $k \leftarrow l$ 
8:   while  $i \leq m \wedge j \leq r$  do
9:     if  $B[i - l] \leq B[j - l]$  then  $A[k] \leftarrow B[i - l]$ ;  $i \leftarrow i + 1$ 
10:    else  $A[k] \leftarrow B[j - l]$ ;  $j \leftarrow j + 1$ 
11:    end if;  $k \leftarrow k + 1$ 
12:   end while
13:   while  $i \leq m$  do  $A[k] \leftarrow B[i - l]$ ;  $i \leftarrow i + 1$ ;  $k \leftarrow k + 1$ 
14:   end while
15:   while  $j \leq r$  do  $A[k] \leftarrow B[j - l]$ ;  $j \leftarrow j + 1$ ;  $k \leftarrow k + 1$ 
16:   end while
17: end procedure
```

Algorithm 38 堆排序规约

```
1: procedure HEAPSORT( $A$ )
2:   BuildMaxHeap( $A$ )
3:   for  $i \leftarrow |A| - 1$  downto 1 do
4:     SWAP( $A[0], A[i]$ ); SiftDown( $A, 0, i$ )
5:   end for
6: end procedure
```

▷ 自底向上 $O(n)$

Algorithm 39 计数排序规约

```
1: procedure COUNTINGSORT( $A, k$ )
2:    $C[0..k] \leftarrow 0$ ;
3:   for all  $x \in A$  do  $C[x] \leftarrow C[x] + 1$ 
4:   end for
5:   for  $i \leftarrow 1$  to  $k$  do  $C[i] \leftarrow C[i] + C[i - 1]$ 
6:   end for
7:    $B \leftarrow A$ ;
8:   for  $i \leftarrow |A| - 1$  downto 0 do  $C[B[i]] \leftarrow C[B[i]] - 1$ ;  $A[C[B[i]]] \leftarrow B[i]$ 
9:   end for
10: end procedure
```

Algorithm 40 桶排序规约

```
1: procedure BUCKETSORT( $A, k$ )
2:   将  $[Min, Max]$  划分为  $k$  个区间  $B_0..B_{k-1}$ 
3:   for all  $x \in A$  do 将  $x$  插入  $B_{\lfloor k \cdot (x - Min) / (Max - Min + 1) \rfloor}$ 
4:   end for
5:   for  $i \leftarrow 0$  to  $k - 1$  do 排序  $B_i$  ▷ 桶内插排
6:   end for
7:   串联  $B_0, \dots, B_{k-1}$  回  $A$ 
8: end procedure
```

Algorithm 41 基数排序规约

```
1: procedure RADIXSORT( $A, D$ )
2:   for  $d \leftarrow 0$  to  $D - 1$  do
3:     按第  $d$  位稳定计数排序  $A$  ▷ 低位到高位
4:   end for
5: end procedure
```

7.3 查找与并查集

7.3.1 二分查找 (Binary Search)

- 对象：有序数组 A 上区间 $[l, r]$, $m = \lfloor (l + r)/2 \rfloor$ 。
- 不变式： $A[m] = x \implies$ 找到； $A[m] < x \implies l = m + 1$ ； 否则 $r = m - 1$ 。
- 定理：时间 $O(\log n)$ ，需随机访问 $O(1)$ （顺序存储实例）。

Algorithm 42 二分查找规约

```
1: procedure BINARYSEARCH( $A, x$ )
2:    $l \leftarrow 0; r \leftarrow |A| - 1$ 
3:   while  $l \leq r$  do
4:      $m \leftarrow \lfloor (l + r)/2 \rfloor$ 
5:     if  $A[m] = x$  then return  $m$ 
6:     else if  $A[m] < x$  then  $l \leftarrow m + 1$ 
7:     elser  $\leftarrow m - 1$ 
8:     end if
9:   end while
10:  return  $-1$ 
11: end procedure
```

7.3.2 插值查找 (Interpolation Search)

- 对象：有序且近似均匀分布的数组 A ，按值的比例估计目标位置。
- 不变式： $m = l + \left\lfloor \frac{x - A[l]}{A[r] - A[l]}(r - l) \right\rfloor$ ，据 $A[m]$ 与 x 的比较收缩区间。
- 定理：均匀分布下期望时间 $O(\log \log n)$ ；分布偏斜时退化至 $O(n)$ 。需 $A[r] \neq A[l]$ 以防除零。

Algorithm 43 插值查找规约

```
1: procedure INTERPOLATIONSEARCH( $A, x$ )
2:    $l \leftarrow 0; r \leftarrow |A| - 1$ 
3:   while  $l \leq r \wedge A[l] \leq x \leq A[r]$  do
4:     if  $A[r] = A[l]$  then
5:        $m \leftarrow l$ 
6:     else  $m \leftarrow l + \lfloor (x - A[l])(r - l)/(A[r] - A[l]) \rfloor$ 
7:     end if
8:     if  $A[m] = x$  then return  $m$ 
9:     else if  $A[m] < x$  then  $l \leftarrow m + 1$ 
10:    elser  $\leftarrow m - 1$ 
11:    end if
12:  end while
13:  return  $-1$ 
14: end procedure
```

7.3.3 分块查找 (Block Search)

- 对象：数组分块，块内无序、块间有序；对每块取关键字上界建索引表。
- 操作：先对索引表块间定位（顺序或二分），再在目标块内顺序查找。
- 定理：设块数 b 、块长 s 、 $n = bs$ ，平均查找长度 $ASL \approx \frac{b+1}{2} + \frac{s+1}{2}$ ，取 $s = \sqrt{n}$ 时最小，为 $O(\sqrt{n})$ 。

Algorithm 44 分块查找规约

```
1: procedure BLOCKSEARCH( $A, x, idx, s$ )
2:    $k \leftarrow$  索引表中最小的  $k$  使  $x \leq idx[k]$  ▷ 块间定位
3:   if  $k$  不存在 then return  $-1$ 
4:   end if
5:   for  $i \leftarrow k \cdot s$  to  $\min(|A| - 1, (k + 1)s - 1)$  do ▷ 块内顺序
6:     if  $A[i] = x$  then return  $i$ 
7:     end if
8:   end for
9:   return  $-1$ 
10: end procedure
```

7.3.4 并查集 (Union-Find)

- 对象：不相交集合族 $\mathcal{S} = \{S_1, \dots, S_k\}$ 。
- 操作：Find(x) 取代表元，Union(x, y) 合并；按秩合并 + 路径压缩后摊还 $O(\alpha(n))$ 。

Algorithm 45 并查集规约（路径压缩 + 按秩合并）

```
1: procedure FIND( $x$ )
2:   if  $parent[x] \neq x$  then  $parent[x] \leftarrow$  Find( $parent[x]$ )
3:   end if
4:   return  $parent[x]$ 
5: end procedure
6: procedure UNION( $x, y$ )
7:    $rx \leftarrow$  Find( $x$ );  $ry \leftarrow$  Find( $y$ )
8:   if  $rx = ry$  then return
9:   end if
10:  if  $rank[rx] < rank[ry]$  then  $parent[rx] \leftarrow ry$ 
11:  else if  $rank[rx] > rank[ry]$  then  $parent[ry] \leftarrow rx$ 
12:  else  $parent[ry] \leftarrow rx$ ;  $rank[rx] \leftarrow rank[rx] + 1$ 
13:  end if
14: end procedure
```

7.4 图论核心算法

7.4.1 最短路径算法

维护 $D : V \rightarrow \mathbb{R} \cup \{\infty\}$, 松弛: $D[u] + w(u, v) < D[v] \implies D[v] \leftarrow D[u] + w(u, v)$ 。

1. **Dijkstra:** 每次取 $D[u]$ 最小的 $u \in V \setminus S$ 入 S 并松弛。不支持负权; 优先队列 ExtractMin/DecreaseKey 为其贪心实例化。
2. **Bellman-Ford:** $|V| - 1$ 轮全量松弛; 第 $|V|$ 轮可松弛 $\implies$ 负环, $O(|V||E|)$ 。
3. **Floyd-Warshall:** $D^{(k)}[i][j] = \min(D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j])$, $O(|V|^3)$, 支持负权, $D[i][i] < 0 \implies$ 负环。

Algorithm 46 Dijkstra 规约 (Q 为优先队列实例)

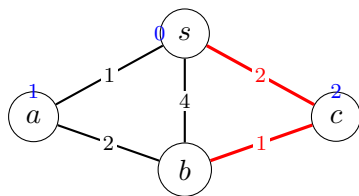
```
1: 输入:  $G = (V, E), s, w; \forall v, D[v] \leftarrow \infty, D[s] \leftarrow 0; S \leftarrow \emptyset; Q \leftarrow V$ 
2: while  $Q \neq \emptyset$  do
3:    $u \leftarrow \text{ExtractMin}(Q); S \leftarrow S \cup \{u\}$ 
4:   for all  $v \in N(u)$  do
5:     if  $D[u] + w(u, v) < D[v]$  then  $D[v] \leftarrow D[u] + w(u, v); \text{DecreaseKey}(Q, v, D[v])$ 
6:   end if
7: end for
8: end while
```

Algorithm 47 Bellman-Ford 规约

```
1: 输入:  $G = (V, E), s, w; \forall v, D[v] \leftarrow \infty, D[s] \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $|V| - 1$  do
3:   for all  $(u, v) \in E$  do
4:     if  $D[u] + w(u, v) < D[v]$  then  $D[v] \leftarrow D[u] + w(u, v); \text{parent}[v] \leftarrow u$ 
5:   end if
6: end for
7: end for
8: for all  $(u, v) \in E$  do
9:   if  $D[u] + w(u, v) < D[v]$  then return “存在负环”
10: end if
11: end for
12: return  $D$ 
```

Algorithm 48 Floyd-Warshall 规约

```
1:  $D^{(0)}[i][j] \leftarrow w(i, j)$  若  $(i, j) \in E$  否则  $\infty, D^{(0)}[i][i] \leftarrow 0$ 
2: for  $k \leftarrow 1$  to  $|V|$  do
3:   for  $i \leftarrow 1$  to  $|V|$  do
4:     for  $j \leftarrow 1$  to  $|V|$  do
5:        $D^{(k)}[i][j] \leftarrow \min(D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j])$ 
6:     end for
7:   end for
8: end for
9: 若  $\exists i, D^{(|V|)}[i][i] < 0$  则存在负环
```



松弛 $D[b] := \min(4, D[c] + w(c, b)) = \min(4, 3) = 3$

图 7.4: Dijkstra 松弛（红边为已定的最短路径树，蓝值为 D ）

7.4.2 最小生成树 (MST)

1. **Prim:** 任意 $r \in V$, 维护 T , 取横切边最小者入 T （割定理）, $O(|E| \log |V|)$ 。
2. **Kruskal:** E 升序, 依序取 (u, v) 若 $\text{Find}(u) \neq \text{Find}(v)$ 则入 MST（并查集实例）, $O(|E| \log |E|)$ 。

Algorithm 49 Prim 规约（ Q 为优先队列）

```

1:  $key[v] \leftarrow \infty, parent[v] \leftarrow \text{nil}, inT[r] \leftarrow \text{true}, key[r] \leftarrow 0, Q \leftarrow V$ 
2: while  $Q \neq \emptyset$  do
3:    $u \leftarrow \text{ExtractMin}(Q); inT[u] \leftarrow \text{true}$ 
4:   for all  $v \in N(u)$  do
5:     if  $\neg inT[v] \wedge w(u, v) < key[v]$  then  $key[v] \leftarrow w(u, v); parent[v] \leftarrow u; \text{DecreaseKey}(Q, v, key[v])$ 
6:     end if
7:   end for
8: end while

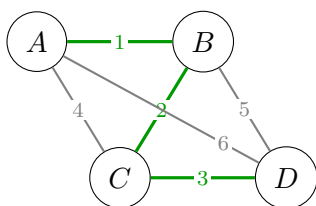
```

Algorithm 50 Kruskal 规约（并查集判环）

```

1:  $A \leftarrow \emptyset$ ; 将  $E$  按  $w$  升序; 初始化并查集
2: for all  $(u, v) \in E$  有序 do
3:   if  $\text{Find}(u) \neq \text{Find}(v)$  then  $A \leftarrow A \cup \{(u, v)\}; \text{Union}(u, v)$ 
4:   end if
5: end for
6: return  $A$ 

```



绿边为 MST: $|V| - 1 = 3$ 条, 总权 6

图 7.5: Kruskal: 按权升序取不构成环的边（灰边被弃）

7.4.3 拓扑排序：Kahn 算法

- 对象：DAG 的入度 $\text{indeg}(v) = |\{u \mid (u, v) \in E\}|$ 。
- 规约： $Q = \{v \mid \text{indeg}(v) = 0\}$ ；弹出 u 入 σ ， $\forall v \in N(u), \text{indeg}(v) - 1$ 为 0 则入 Q ； $|\sigma| < |V| \implies$ 有环。

Algorithm 51 Kahn 拓扑排序规约

```
1: procedure KAHN( $G$ )
2:   计算  $\text{indeg}[v]$ ;  $Q \leftarrow \{v \mid \text{indeg}[v] = 0\}$ ;  $\sigma \leftarrow \langle \rangle$ 
3:   while  $Q \neq \emptyset$  do
4:      $u \leftarrow \text{pop}(Q)$ ;  $\sigma \leftarrow \sigma \circ \langle u \rangle$ 
5:     for all  $v \in N(u)$  do  $\text{indeg}[v] \leftarrow \text{indeg}[v] - 1$ ;
6:       if  $\text{indeg}[v] = 0$  then  $Q \leftarrow Q \cup \{v\}$ 
7:     end if
8:   end for
9: end while
10: if  $|\sigma| < |V|$  then return “有环”
11: elsereturn  $\sigma$ 
12: end if
13: end procedure
```

7.4.4 启发式搜索：A* (A-Star)

- 对象：加权图 G ，起点 s ，终点 t ，启发式 $h: V \rightarrow \mathbb{R}_{\geq 0}$ 估计 $v \rightarrow t$ 代价。
- 不变式： $g(v)$ 为 $s \rightarrow v$ 已知最短， $f(v) = g(v) + h(v)$ 为优先键；可采纳 $h(v) \leq h^*(v)$ （真实最优），一致 $h(u) \leq w(u, v) + h(v)$ 。
- 定理：可采纳时 A* 首次出队 t 即最优； $h \equiv 0$ 退化为 Dijkstra；一致时无需重开集。

Algorithm 52 A* 规约

```
1:  $g[s] \leftarrow 0, f[s] \leftarrow h(s), Q \leftarrow \{s\}$  ▷  $Q$  按  $f$  优先
2: while  $Q \neq \emptyset$  do
3:    $u \leftarrow \text{ExtractMin}(Q)$  ▷ 按  $f$ 
4:   if  $u = t$  then return 重构路径
5:   end if
6:   for all  $v \in N(u)$  do
7:      $ng \leftarrow g[u] + w(u, v)$ 
8:     if  $ng < g[v]$  then  $g[v] \leftarrow ng; f[v] \leftarrow ng + h(v); \text{parent}[v] \leftarrow u; Q \leftarrow Q \cup \{v\}$ 
9:   end if
10: end for
11: end while
```

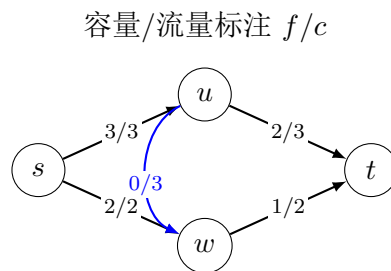
7.5 高级工程专项算子

7.5.1 网络流 (Network Flow)

- 对象: $G = (V, E)$, 容量 $c: E \rightarrow \mathbb{R}^+$, 流 $f: E \rightarrow \mathbb{R}$ 满足 $0 \leq f \leq c$ 且 $\forall u \neq s, t, \sum_v f(u, v) = 0$ 。
- 残量: $c_f(u, v) = c(u, v) - f(u, v)$, 增广路 P 在 G_f 中 $s \rightarrow t$ 。
- 定理 (最大流最小割): $|f|_{\max} = \min_{S \ni s, T \ni t} c(S, T)$; 增广至无 $s \rightarrow t$ 路即最优。

Algorithm 53 Dinic 规约 (分层 + 阻塞流)

```
1: procedure DINIC( $G, s, t$ )
2:    $f \leftarrow 0$ 
3:   while 以  $c_f$  BFS 构分层  $level[]$  且可达  $t$  do
4:      $it[v] \leftarrow 0$  对所有  $v$ 
5:     while  $push \leftarrow \text{DFS}(s, t, \infty)$  推得  $> 0$  do  $f \leftarrow f + push$ 
6:     end while
7:   end while
8:   return  $f$ 
9: end procedure
10: procedure DFS( $u, t, upTo$ )
11:   if  $u = t$  then return  $upTo$ 
12:   end if
13:   for  $i \leftarrow it[u]$  to  $|Adj[u]| - 1$  do
14:      $it[u] \leftarrow i$ ; 令  $(u, v) \leftarrow Adj[u][i]$ 
15:     if  $c_f(u, v) > 0 \wedge level[v] = level[u] + 1$  then
16:        $d \leftarrow \text{DFS}(v, t, \min(upTo, c_f(u, v)))$ 
17:       if  $d > 0$  then  $c_f(u, v) \leftarrow c_f(u, v) - d$ ;  $c_f(v, u) \leftarrow c_f(v, u) + d$ ; return  $d$ 
18:     end if
19:   end for
20:   return 0
21: end procedure
```



分层 $s \rightarrow \{u, w\} \rightarrow t$; 蓝为反向残量边 (可退流)

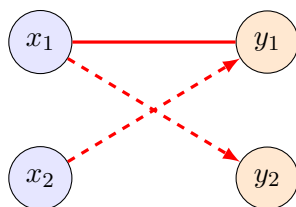
图 7.6: 网络流残量网络与分层图 (Dinic)

7.5.2 二分图最大匹配

- 对象: $G = (X \cup Y, E)$, 匹配 $M \subseteq E$ 无公共点。
- 定理 (Berge): M 最大 $\iff$ 无增广路 P (端点未匹配、交替)。 $M' \leftarrow M \oplus P$ 增大。

Algorithm 54 匈牙利增广规约

```
1: procedure MAXMATCHING( $G$ )
2:    $M \leftarrow \emptyset$ 
3:   for all  $u \in X$  do
4:      $vis[ ] \leftarrow \text{false}$ 
5:     if DFS( $u$ ) then ▷ 找到增广则  $M$  已在 DFS 内翻转
6:       end if
7:   end for
8:   return  $M$ 
9: end procedure
10: procedure DFS( $u$ )
11:   for all  $v \in N(u)$  do
12:     if  $\neg vis[v]$  then  $vis[v] \leftarrow \text{true}$ 
13:       if  $match[v] = \text{nil} \vee \text{DFS}(match[v])$  then  $match[v] \leftarrow u$ ;  $M \leftarrow M \oplus (u, v)$ ; return true
14:       end if
15:     end if
16:   end for
17:   return false
18: end procedure
```



实红 = 匹配 $M = \{x_1y_1\}$; 虚红 = 增广路 $x_2 - y_1 - x_1 - y_2$
翻转后 $M' = \{x_2y_1, x_1y_2\}$, $|M'| = |M| + 1$

图 7.7: 二分图最大匹配: 沿增广路翻转 (Berge 定理)

7.5.3 强连通分量 (SCC): Tarjan 算法

- 对象: DFS 时间戳 $DFN[u]$, 追溯 $LOW[u]$ 。
- 不变式: u 入栈; v 未访则 $LOW[u] = \min(LOW[u], LOW[v])$; v 在栈则 $LOW[u] = \min(LOW[u], DFN[v])$ 。
- 定理: $LOW[u] = DFN[u] \iff u$ 为 SCC 根, 栈中 u 及以上一次弹出构成一 SCC, $O(|V| + |E|)$ 。

Algorithm 55 Tarjan SCC 规约

```
1:  $timer \leftarrow 0$ ; 栈  $S \leftarrow \emptyset$ 
2: procedure TARJAN( $u$ )
3:    $DFN[u] \leftarrow LOW[u] \leftarrow timer \leftarrow timer + 1$ ;  $S.push(u)$ ;  $onStack[u] \leftarrow \text{true}$ 
4:   for all  $v \in N(u)$  do
5:     if  $DFN[v] = \text{nil}$  then Tarjan( $v$ );  $LOW[u] \leftarrow \min(LOW[u], LOW[v])$ 
6:     else if  $onStack[v]$  then  $LOW[u] \leftarrow \min(LOW[u], DFN[v])$ 
7:     end if
8:   end for
9:   if  $LOW[u] = DFN[u]$  then
10:    repeat  $v \leftarrow S.pop()$ ;  $onStack[v] \leftarrow \text{false}$ ; 输出  $v$  归入同一 SCC
11:    until  $v = u$ 
12:   end if
13: end procedure
```

第八章 算法对比

本章以表汇集核心算子的时空复杂度与关键特性，便于对比与选型。

8.1 为什么要比较算法：多维权衡

渐近复杂度只是评价算法的第一把尺子。理论上的 $O(f(n))$ 告诉我们在 n 趋于无穷时的增长趋势，却忽略了常数因子、输入分布、内存层次与实现细节。真实的工程选型必须同时在 **时间**、**空间**、**稳定性** 与 **适用场景** 四个维度上权衡，任何一个维度的放松或收紧都可能改变最优解。

8.1.1 四个权衡维度

- **时间**：需要区分 最好、平均、最坏三种界。例如快速排序平均为 $O(n \log n)$ 、最坏退化为 $O(n^2)$ ，而堆排序最坏仍有 $O(n \log n)$ 保证；只记住一个数字会误导选型。
- **空间**：原地算法辅助空间为 $O(1)$ ，归并需要 $O(n)$ 副本，散列需要 $O(n)$ 桶数组。当内存受限或数据驻留外存时，空间维度可能直接否决某算法。
- **稳定性**：稳定算法保持相等键的原始相对次序。多关键字排序中，先按次关键字排、再按主关键字稳定排序即可得到等价于复合键排序的结果；不稳定算法会破坏这一性质。
- **适用场景**：数据规模 n 、值域宽度 k 、是否近似有序、是否为整数键、是静态一次查找还是动态频繁更新，都会改变最优选择。

8.1.2 渐近界与常数因子

渐进分析刻意忽略常数：若 $T(n) = O(f(n))$ ，则存在常数 c 使 $T(n) \leq c f(n)$ 。但在有限规模下， $O(n \log n)$ 的算法完全可能比 $O(n^2)$ 慢，因为其常数因子更大或访存模式更差。

$$T(n) = O(f(n)) \iff \exists c > 0, n_0 > 0, \forall n \geq n_0 : T(n) \leq c f(n). \quad (8.1)$$

此外，理论模型假设每次基本操作等代价，而现代 CPU 的缓存命中、分支预测与向量化会显著改变真实运行时间。因此本章的对比表是选型的起点，不是终点。

8.2 排序算法综合对比

设输入 $A = \langle a_1, \dots, a_n \rangle$ 需重排为 $a'_1 \leq a'_2 \leq \dots \leq a'_n$ 。表 8.1 汇总十大排序算法在三个时间界、空间与稳定性上的表现。

算法	最好	平均	最坏	空间	稳定性	关键特征
冒泡	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	相邻交换消逆序, 已序时提前终止
选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定	比较次数固定 $\frac{n(n-1)}{2}$
插入	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	近似有序时接近线性
希尔	$O(n \log n)$	$O(n^{1.3})$	$O(n^2)$	$O(1)$	不稳定	增量序列决定, 分组插入
归并	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定	严格分治合并, 需辅助数组
快速	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	不稳定	平均最快, 随机化基准可避退化
堆排	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定	原地且最坏有保证
计数	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	稳定	整数键, 突破比较下界
基数	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$	稳定	低位到高位, d 为位数
桶排	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n+k)$	稳定	均匀分布近线性, 依赖桶内排序

表 8.1: 十大排序算法综合对比

8.2.1 比较类排序的下界

任何仅依赖关键字两两比较的排序算法, 其判定树是一棵二叉树叶数不少于 $n!$ 的二叉树, 树高即最坏比较次数, 故有

$$T_{\text{cmp}}(n) \geq \lceil \log_2(n!) \rceil = \Theta(n \log n). \quad (8.2)$$

由 $\log_2(n!) = \sum_{i=1}^n \log_2 i \geq \sum_{i=\lceil n/2 \rceil}^n \log_2 \frac{n}{2} \geq \frac{n}{2} \log_2 \frac{n}{2} = \Omega(n \log n)$ 即得。归并、堆排序都达到该下界, 故是比较类算法的最优阶。

8.2.2 分治排序的递推

以归并排序为例, 每次折半递归并线性合并, 由主定理第二情形:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) \implies T(n) = \Theta(n \log n). \quad (8.3)$$

8.2.3 非比较排序的线性界

计数排序通过值域寻址绕过比较下界, 基数排序对每一位复用计数排序:

$$T_{\text{count}}(n, k) = \Theta(n+k), \quad T_{\text{radix}}(n, d, k) = \Theta(d(n+k)). \quad (8.4)$$

当 $k = O(n)$ 且位数 d 为常数时, 两者均为 $O(n)$ 。代价是必须知道键的整数值域, 并对空间 $O(k)$ 或 $O(n+k)$ 付费。

8.3 查找算法对比

查找问题是在数据集 S 中定位关键字等于 x 的元素。不同算法的前提条件差异极大，表 8.2 给出对比。

算法	前提条件	平均/期望	最坏	适用场景
顺序查找	无	$O(n)$	$O(n)$	无序或小规模，链式存储
二分查找	顺序存储且有序	$O(\log n)$	$O(\log n)$	静态有序表，随机访问
插值查找	有序且近似均匀分布	$O(\log \log n)$	$O(n)$	键均匀稠密，值域可比例定位
分块查找	块间有序、块内无序	$O(\sqrt{n})$	$O(\sqrt{n})$	顺序与链式的折中，便于动态插入
散列查找	散列函数与冲突处理	$O(1)$ 期望	$O(n)$	仅等值查询，无需有序

表 8.2: 查找算法对比

8.3.1 平均查找长度

顺序存储在等概率假设下的平均查找长度（ASL）为

$$\text{ASL}_{\text{seq}} = \frac{n+1}{2}, \quad \text{ASL}_{\text{bin}} \approx \log_2(n+1) - 1, \quad \text{ASL}_{\text{blk}} = \frac{b+1}{2} + \frac{s+1}{2} \quad (n = bs). \quad (8.5)$$

分块查找在 $s = \sqrt{n}$ 时取得最小 $\text{ASL} = O(\sqrt{n})$ 。散列表的期望探测次数由装载因子 $\alpha = n/m$ (n 个元素、 m 个槽) 决定：

$$E[\text{probes}]_{\text{chain}} \approx \frac{1}{\alpha} \ln \frac{1}{1-\alpha}, \quad E[\text{probes}]_{\text{linear}} \approx \frac{1}{2} \left(1 + \frac{1}{1-\alpha} \right). \quad (8.6)$$

可见散列的 $O(1)$ 是期望意义下的常数， $\alpha \rightarrow 1$ 时探测数急剧上升。

8.4 增长曲线与计数级差异

8.4.1 增长曲线的可视化

图 8.1 采用双对数坐标，将四种典型增长函数放在 $n = 10^2 \sim 10^5$ 区间内比较。纵轴为基本操作次数，可直接读出不同算法之间的数量级差异。

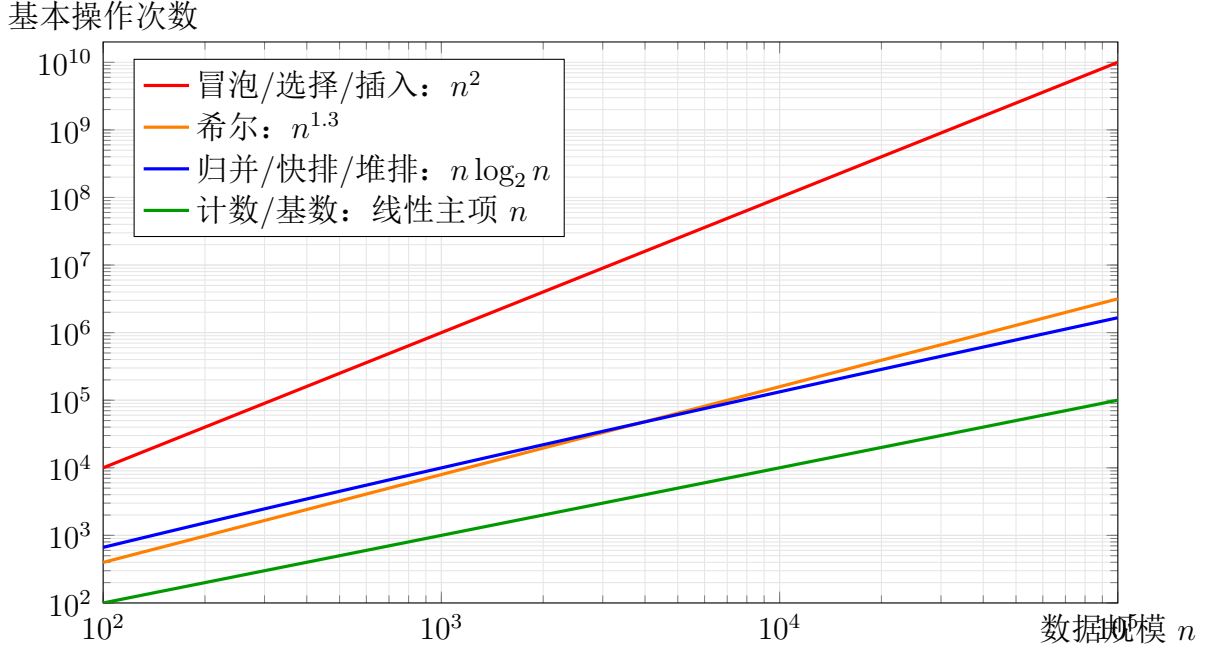


图 8.1: 各排序算法基本操作次数的增长对比（双对数坐标， $n = 10^2 \sim 10^5$ ）

在 $n = 10^5$ 处， $n^2 = 10^{10}$ 、 $n^{1.3} \approx 3.2 \times 10^6$ 、 $n \log_2 n \approx 1.7 \times 10^6$ ，三者跨越了约四个数量级。

8.4.2 计数级差异算例（ $n = 10^5$ ）

设一次关键字比较耗时为常数 $t_c = 10^{-9}$ s（约每秒 10^9 次比较），针对 $n = 10^5$ 的随机排列数组估算选择排序与归并排序的开销。

选择排序比较次数固定为

$$T_{\text{sel}}(n) = \frac{n(n-1)}{2} \approx \frac{10^5 \times (10^5 - 1)}{2} \approx 5.0 \times 10^9, \quad t_{\text{sel}} \approx 5.0 \text{ s}. \quad (8.7)$$

归并排序比较次数约为 $n \log_2 n - n$ ：

$$T_{\text{merge}}(n) \approx n \log_2 n - n \approx 10^5 \times 16.6 - 10^5 \approx 1.56 \times 10^6, \quad t_{\text{merge}} \approx 1.6 \text{ ms}. \quad (8.8)$$

两者时间之比与 n 近似线性增长：

$$\begin{aligned} \frac{T_{\text{sel}}}{T_{\text{merge}}} &= \frac{n(n-1)/2}{n \log_2 n - n} \approx \frac{n-1}{2 \log_2 n} \\ &\approx \frac{10^5}{2 \times 16.6} \approx 3.0 \times 10^3. \end{aligned} \quad (8.9)$$

即选择排序约慢 3000 倍。若改用基数排序（32 位键、每趟 8 位， $d = 4$ 、 $k = 256$ ），移动次数 $d(n + k) \approx 4.0 \times 10^5$ ，在同等常数下比归并再快数倍，代价是额外的 $O(n + k)$ 桶空间。

8.5 时间-空间权衡

空间与时间可以互相转化：多花内存常能少花时间，反之亦然。表 8.3 归纳典型策略。

策略	代表算法	辅助空间	代价与收益
原地排序	冒泡/选择/插入/希尔/堆排	$O(1)$	省内存，但多不稳定或最坏退化
递归栈开销	快速/归并（自顶向下）	$O(\log n) \sim O(n)$	分治简洁，栈深依赖划分平衡度
线性辅助	归并	$O(n)$	换取稳定性与严格 $O(n \log n)$
空间换时间	计数/基数/桶排	$O(k)$ 或 $O(n + k)$	突破 $\Omega(n \log n)$ ，依赖键值域
散列换时间	散列表	$O(n)$	查找由 $O(n)$ 降至 $O(1)$ 期望，付出冲突代价

表 8.3: 时间-空间权衡策略

8.5.1 原地与非原地

原地（in-place）算法仅使用 $O(1)$ （或含递归栈 $O(\log n)$ ）辅助空间，在原数组上直接交换；非原地算法需要与输入等大的副本。归并排序的非原地性换来稳定与最坏保证；快速排序的原地划分带来优秀缓存局部性，但递归栈与最坏退化需要关注。快速排序可通过“尾递归消除 + 先递归较短一侧”将栈深压到 $O(\log n)$ 。

8.5.2 以空间换时间的边界

空间换时间并非无条件成立：计数排序的空间 $O(k)$ 在值域 $k \gg n$ 时不可接受；散列的 $O(n)$ 桶在装载因子趋近 1 时期望探测数发散。选型时须同时检查时空两个约束是否都可满足。

8.6 如何选择算法：决策清单

8.6.1 排序算法选择

- 数据规模： $n \leq 50$ 时插入排序常数小、实际最快；大 n 再考虑 $O(n \log n)$ 。
- 是否近似有序： 接近有序时插入或希尔接近线性。
- 稳定性要求： 多关键字场景优先归并、插入、计数与基数。
- 内存约束： 内存紧张选堆排（原地且最坏有保证）或快排（原地、平均最快）。
- 键的类型： 整数且值域 $k = O(n)$ 时，计数/基数/桶排可突破比较下界。
- 最坏时间要求： 实时系统要求最坏有保证，则弃快排取堆排或归并。

8.6.2 查找算法选择

- 1. 数据是否有序？无序则顺序查找或先排序（动态场景用散列）。
- 2. 是否需要随机访问与二分？链式存储无法二分，只能顺序或转跳表。
- 3. 键分布是否均匀？均匀且稠密可用插值查找。
- 4. 是否只需等值查询？是则散列最优，但要评估装载因子与冲突处理。
- 5. 是否频繁动态插入删除？分块或平衡树优于静态有序数组。

图 8.2 给出排序选型的快速决策路径。

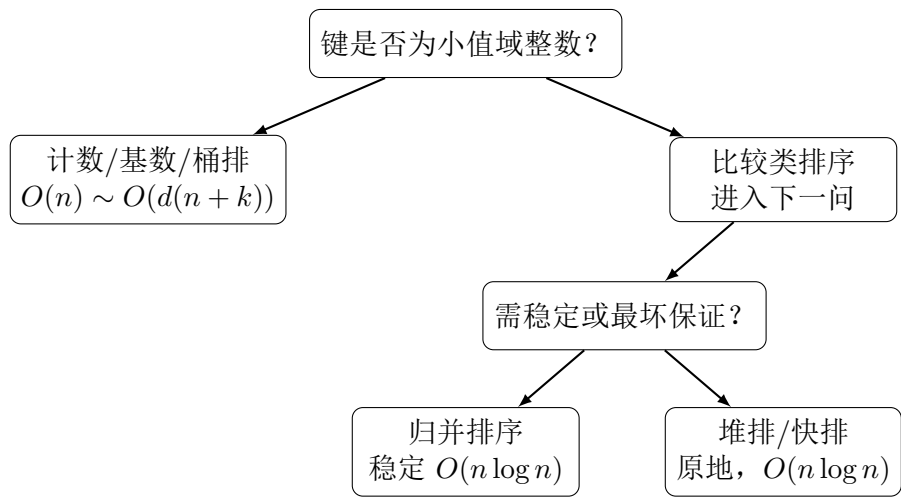


图 8.2: 排序算法选择决策路径

8.7 其他算子横向对比

8.7.1 高级图论算子

算法	目标	时间	空间	适用与限制
并查集	集合维护	$O(\alpha(n))$ 摊还	$O(n)$	近 $O(1)$ ，支持动态合并
Dijkstra	单源最短路	$O(E \log V)$	$O(V)$	不支持负权边
A*	单源启发搜索	$O(E)$ 期望	$O(V)$	启发 h 可采纳则最优
Bellman-Ford	单源带权最短路	$O(V E)$	$O(V)$	支持负权，可判负环
Floyd-Warshall	多源最短路	$O(V ^3)$	$O(V ^2)$	支持负权， $D[i][i] < 0$ 判负环
Prim	最小生成树	$O(E \log V)$	$O(V)$	稠密图较优
Kruskal	最小生成树	$O(E \log E)$	$O(E)$	稀疏图较优，依赖并查集
Kahn	拓扑排序	$O(V + E)$	$O(V)$	输出不足 $ V $ 则有环
Tarjan	强连通分量	$O(V + E)$	$O(V)$	单次 DFS， $LOW = DFN$ 为根
Dinic	最大流	$O(V ^2 E)$	$O(V + E)$	残量网络无增广路即最优

表 8.4: 高级图论与检索算子对比

8.7.2 树算子

算子	目标	时间	空间	核心依赖与限制
二叉树遍历	全树访问	$O(n)$	$O(H)$	递归/显式栈，线索化可降 $O(1)$
BST	有序维护与检索	$O(H)$	$O(n)$	偏斜时退化至 $O(n)$
AVL	平衡检索维护	$O(\log n)$	$O(n)$	平衡因子与旋转
红黑树	近似平衡维护	$O(\log n)$	$O(n)$	黑高与变色，插入至多两次旋转
B/B+ 树	外存索引	$O(\log_m n)$	$O(n)$	叶层等深，适合磁盘
Huffman	最优前缀码	$O(n \log n)$	$O(n)$	最小堆贪心，需已知频次
堆	最值维护	建堆 $O(n)$ ，操作 $O(\log n)$	$O(n)$	完全二叉树，上滤/下滤
LCA 倍增	树上祖先查询	$O(\log V)$	$O(V \log V)$	预处理 $up[v][j]$

表 8.5: 树算子对比

8.8 本章小结

算法对比的核心结论可以概括为三点：

- 没有“最好”的算法，只有在给定约束下最合适的算法。时间、空间、稳定性与适用场景缺一不可。
- 比较类排序存在 $\Omega(n \log n)$ 下界，达到该界的归并、堆排是通用最优；当键为小值域整数时，计数/基数/桶排可用空间换时间降到线性。
- 查找算法高度依赖前提条件：有序支持二分与插值，等值查询优先散列，动态更新则考虑分块与平衡树。

8.9 常见误区

- 误区一：**认为 $O(n \log n)$ 在任何规模下都快于 $O(n^2)$ 。小规模时低阶算法常数更小，反而更快；须结合 n 与常数因子判断。
- 误区二：**把平均复杂度当作最坏复杂度。快排平均 $O(n \log n)$ ，但对已序数据用固定基准会退化到 $O(n^2)$ 。
- 误区三：**认为 $O(1)$ 空间就是“不占额外内存”。它指不随 n 增长，常量级辅助仍存在，递归算法的栈深往往并非 $O(1)$ 。
- 误区四：**认为稳定排序一定更优。稳定性以时间或空间为代价，无需保持次序时应优先考虑速度与内存。
- 误区五：**认为散列查找必然 $O(1)$ 。当装载因子 $\alpha \rightarrow 1$ 或散列函数较差时，期望探测数发散，最坏退化为 $O(n)$ 。
- 误区六：**忽略缓存与访存局部性。相邻访问的原地快排常胜过理论更优但跳访的算法；渐近界不等于真实墙钟时间。